HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

BÁO CÁO PROJECT 01

ĐỀ TÀI: DIANEXUS – HỆ ĐA TÁC TỬ SÀNG LỌC, TƯ VẤN VÀ TRA CỨU TRI THỨC ĐÁI THÁO ĐƯỜNG TỪ HỌC SÂU, NEO4J, HYBRID RAG, LUẬT VÀ HỌC TĂNG CƯỜNG

Lớp:	D23CTPM01
Nhóm:	Nhóm lớp CT – Nhóm 07
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 11/2026

Mục lục

Danh mục Hình vẽ **iii**

Danh mục Bảng biểu **iv**

1 Tổng quan đề tài **1**
1.1 Bối cảnh và bài toán 1
1.2 Ánh xạ sơ đồ đề bài sang hệ thống 1
1.3 Mục tiêu và đóng góp riêng 2

2 Cơ sở lý thuyết **4**
2.1 Đái tháo đường và tiêu chí chẩn đoán 4
2.2 Học máy trên dữ liệu bảng 4
2.3 Học sâu cho bảng và ảnh 5
2.4 Đồ thị tri thức và RAG 6
2.5 Hệ luật 7
2.6 Học tăng cường 7
2.7 Tác tử và hệ đa tác tử 8

3 Dữ liệu và cơ sở dữ liệu **9**
3.1 Các nguồn dữ liệu 9
3.2 Thiết kế CSDL 9
3.3 Làm sạch và chống rò rỉ 10
3.4 Khám phá dữ liệu 11

4 Học máy, học sâu và dự đoán **14**
4.1 Quy trình thí nghiệm 14
4.2 Mô hình sâu cho dữ liệu bảng 14
4.3 Kết quả trên tập kiểm tra 15
4.4 Đối chuẩn chéo miền 18
4.5 Phân cấp bệnh võng mạc 19
4.6 Phục vụ mô hình 22

5 Cơ sở tri thức, Neo4j và RAG **23**
5.1 Thu thập và tiền xử lý 23
5.2 Ontology và đồ thị trên Neo4j 24
5.3 Truy xuất lai 28

5.4 Đánh giá truy xuất 29

6 Luật, học tăng cường và hệ đa tác tử **31**
6.1 Hệ luật an toàn 31
6.2 Học tăng cường: sàng lọc thích ứng 32
6.2.1 Mô hình hóa 32
6.2.2 Huấn luyện Double DQN 33
6.2.3 Kết quả 35
6.3 Hệ đa tác tử 36
6.3.1 Vai trò và đồ thị trạng thái 36
6.3.2 Critic kiểm bằng mã 38
6.3.3 Đánh giá bằng thí nghiệm cắt bỏ 38

7 Website DiaNexus **42**
7.1 Kiến trúc triển khai 42
7.2 Các trang chức năng 43
7.3 Kiểm thử và an toàn 46

8 Kết luận và hướng phát triển **47**
8.1 Kết quả đạt được 47
8.2 Những gì đồ án cho thấy 47
8.3 Giới hạn 47
8.4 Hướng phát triển 48

Tài liệu tham khảo **49**

Danh mục Hình vẽ

1.1 Sơ đồ đề bài và cách DiaNexus hiện thực từng khối 2

2.1 FT-Transformer: mỗi đặc trưng là một token, token CLS đưa ra dự đoán 5
2.2 Mạng tích chập phân cấp bệnh võng mạc và đường Grad-CAM 6
2.3 Bốn tầng của đồ thị tri thức 6
2.4 Truy xuất lai: ba bộ truy xuất, hợp RRF rồi xếp hạng lại 7
2.5 Sàng lọc thích ứng dưới dạng MDP 8
2.6 Mô hình bảng đen và các vòng phản hồi của hệ đa tác tử 8

3.1 Lược đồ CSDL: bảng dữ liệu và bảng vận hành 10
3.2 Tỉ lệ lớp dương của năm bộ dữ liệu 11
3.3 Tỉ lệ đái tháo đường theo từng mức HbA1c và đường huyết: bước nhảy tới 100% . 12
3.4 Tỉ lệ đái tháo đường tăng theo tuổi và BMI trên hai nguồn độc lập 12
3.5 Mười đặc trưng tương quan mạnh nhất với nhãn trong bảng `brfss` 12
3.6 Tỉ lệ tái nhập viện dưới 30 ngày theo số lần nội trú và chẩn đoán chính 13

4.1 Quy trình chung: mọi lựa chọn diễn ra trên val, test chỉ mở một lần 14
4.2 ROC-AUC của sáu mô hình trên ba bài toán 16
4.3 ROC và Precision–Recall của bài sàng lọc không xét nghiệm 17
4.4 Biểu đồ tin cậy trước và sau hiệu chỉnh 17
4.5 Độ quan trọng SHAP trung bình của hai mô hình phục vụ 18
4.6 ROC-AUC của XGBoost khi huấn luyện ở miền nguồn, chấm ở miền đích 19
4.7 Ảnh đáy mắt thật của năm cấp bệnh 19
4.8 Ma trận nhầm lẫn của EfficientNet-B0 trên test 21
4.9 Grad-CAM: vùng ảnh mà mô hình dựa vào để phân cấp 21

5.1 Đường ống từ tài liệu thô đến chỉ mục và đồ thị 23
5.2 Số cạnh theo từng loại quan hệ trong đồ thị (thang log) 26
5.3 Một phần đồ thị thật quanh đái tháo đường type 2 27
5.4 MRR theo nhóm câu hỏi: khó nhất là câu tiếng Việt trên nguồn tiếng Anh 30

6.1 Đánh đổi giữa chi phí hỏi và AUC của các chính sách 35
6.2 Tỉ lệ người được hỏi từng câu dưới chính sách DQN 36
6.3 Đồ thị trạng thái của hệ đa tác tử, nét đỏ là cạnh quay lui 38
6.4 Tỉ lệ đạt theo từng tiêu chí của năm cấu hình 40

7.1 Bốn tầng triển khai cục bộ của DiaNexus 42
7.2 Trang Tổng quan: số liệu sống và sơ đồ hai nhánh 44
7.3 Trang Hội chẩn: dòng thời gian tác tử trực tiếp và trích dẫn 44
7.4 Trang Sàng lọc thích ứng: chính sách RL chọn từng câu hỏi 45
7.5 Phòng thí nghiệm nguy cơ: what-if, SHAP và song sinh số 45
7.6 Hai trang trực quan của DiaNexus 46
7.7 Hai trang vận hành: dữ liệu và nhật ký 46

Danh mục Bảng biểu

1.1 Ánh xạ khối trên sơ đồ sang thành phần của DiaNexus 2

2.1 Ngưỡng chẩn đoán theo ADA 4

3.1 Năm bộ dữ liệu bảng trong CSDL 9

4.1 Kết quả sáu mô hình trên test của ba bài toán sau hiệu chỉnh isotonic 15
4.2 Kết quả phân cấp võng mạc trên test (chọn mô hình bằng QWK trên val) 20

5.1 Nguồn của cơ sở tri thức 24
5.2 Năm chế độ truy xuất và thí nghiệm cắt bỏ từng bộ khỏi RRF 29
5.3 Sinh câu trả lời có và không có RAG 30

6.1 Nhóm luật của RuleGuard 31
6.2 DQN dừng thích ứng theo λ trên test, so với AUC của hai đối chứng tại cùng chi phí 35
6.3 Tám vai trò của hệ đa tác tử 38
6.4 Thí nghiệm cắt bỏ trên 48 kịch bản, mỗi cấu hình một hàng 40

Chương 1
Tổng quan đề tài

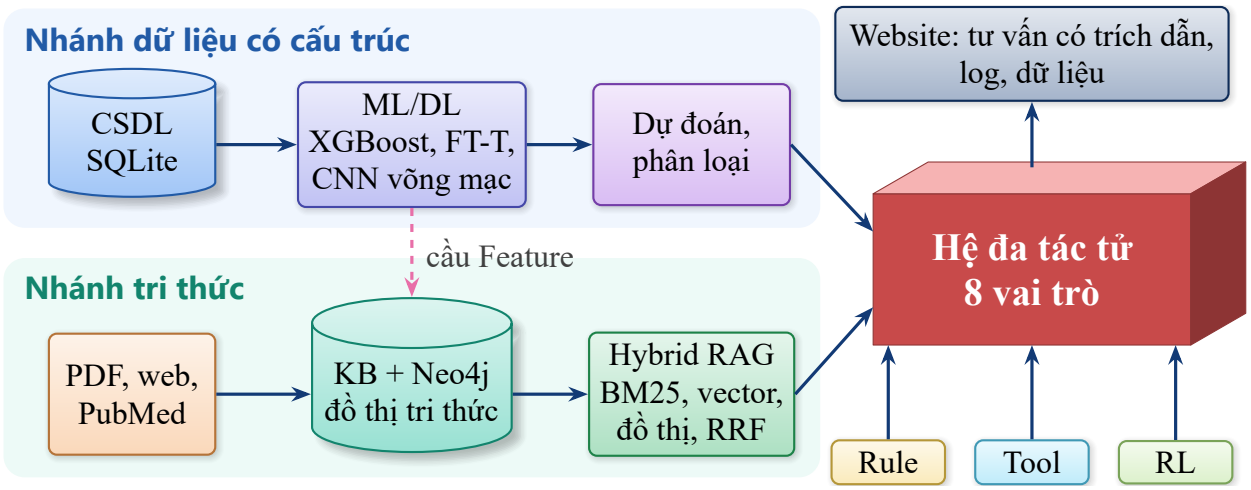
1.1 Bối cảnh và bài toán

Đái tháo đường là bệnh mạn tính tiến triển âm thầm: nhiều người chỉ biết mình mắc bệnh khi đã có biến chứng ở mắt, thận hoặc tim mạch [1]. Muốn hỗ trợ một người dân thật sự, hệ thống phải làm được ba việc rất khác nhau. Việc thứ nhất là ước lượng nguy cơ từ hồ sơ, tức bài toán phân loại trên dữ liệu có cấu trúc. Việc thứ hai là giải thích và tư vấn bằng tri thức y khoa có nguồn rõ ràng, tức bài toán truy xuất. Việc thứ ba là quyết định nên hỏi gì tiếp theo, vì người dùng không có sẵn đủ xét nghiệm và cũng không kiên nhẫn trả lời hàng chục câu.

Một mô hình đơn lẻ chỉ trả về một xác suất, còn một chatbot đơn lẻ dễ nêu con số không có nguồn. Đồ án xây dựng **DiaNexus**, một hệ đa tác tử trong đó mỗi phần việc do một vai trò riêng đảm nhận, kết quả được kiểm tra chéo rồi mới đến tay người dùng. Hệ thống chạy hoàn toàn cục bộ trên máy cá nhân và chỉ mang tính tham khảo, không thay thế chẩn đoán của bác sĩ.

1.2 Ánh xạ sơ đồ đề bài sang hệ thống

Đề bài cho sơ đồ hai nhánh. Nhánh dữ liệu đi từ cơ sở dữ liệu qua học máy, học sâu tới dự đoán và phân loại. Nhánh tri thức đi từ PDF, trang web qua cơ sở tri thức, RAG tới tác tử. Hai nhánh hợp lại tại tác tử, có sự hỗ trợ của luật, công cụ và học tăng cường. Hình 1.1 cho thấy cách DiaNexus hiện thực sơ đồ này, Bảng 1.1 chỉ rõ từng khối nằm ở đâu trong mã nguồn.



Hình 1.1. Sơ đồ đề bài và cách DiaNexus hiện thực từng khối

Bảng 1.1. Ánh xạ khối trên sơ đồ sang thành phần của DiaNexus

Khối	Hiện thực	Mã nguồn
DB	SQLite nhiều bảng	`dnx/db.py`
ML/DL	XGBoost, FT-Transformer, CNN	`dnx/ml/`
Dự đoán	Xác suất hiệu chỉnh, SHAP, what-if	`dnx/ml/serve.py`
PDF/web, KB	Thu thập, chunk, chỉ mục	`dnx/kb/`
Neo4j	Đồ thị bốn tầng	`scripts/kb_graph.py`
RAG	BM25, vector, đồ thị, RRF	`dnx/kb/search.py`
Rule	Suy diễn tiến có nguồn	`dnx/rules.py`
RL	Double DQN sàng lọc thích ứng	`dnx/rl/`
Agent	Tám vai trò, cạnh quay lui	`dnx/agents/`
Website	FastAPI, 3D, SSE	`web/`

1.3 Mục tiêu và đóng góp riêng

Đồ án đặt ra các mục tiêu có thể đo được, mỗi mục tiêu ứng với một phần đánh giá ở các chương sau:

1. Xây dựng CSDL nhiều nguồn và so sánh mô hình cổ điển với mô hình sâu trên ba bài toán: chẩn đoán có xét nghiệm, sàng lọc không xét nghiệm, tái nhập viện.
2. Phân cấp bệnh võng mạc đái tháo đường từ ảnh đáy mắt và giải thích bằng Grad-CAM.
3. Dựng đồ thị tri thức thật trên Neo4j từ nhiều nguồn, kết hợp với truy xuất lai và đo bằng các chỉ số xếp hạng.
4. Huấn luyện chính sách học tăng cường chọn câu hỏi sàng lọc theo từng người, trên dữ liệu thật chứ không phải môi trường giả lập.

5. Chứng minh giá trị của từng thành phần bằng thí nghiệm cắt bỏ trên cùng bộ kịch bản.
6. Triển khai website localhost có theo dõi tác tử trực tiếp, nhật ký và dữ liệu.

Điểm mới của đồ án là **sàng lọc thích ứng**: tác tử học cách đặt ít câu hỏi hơn mà vẫn giữ được chất lượng dự đoán, thứ tự câu hỏi và thời điểm dừng thay đổi theo từng người. Mọi con số trong báo cáo được sinh tự động từ tệp kết quả của lần chạy thực tế, đoạn mã được trích nguyên văn từ mã nguồn đã chạy.

Chương 2
Cơ sở lý thuyết

2.1 Đái tháo đường và tiêu chí chẩn đoán

Đái tháo đường là nhóm rối loạn chuyển hóa có đặc trưng tăng đường huyết mạn tính do thiếu insulin, kháng insulin hoặc cả hai. Típ 2 chiếm đa số và gắn chặt với tuổi, béo phì, ít vận động. Bảng 2.1 tóm tắt các ngưỡng thường dùng theo Hiệp hội Đái tháo đường Hoa Kỳ [2]. Biến chứng vi mạch ở võng mạc là lý do đồ án bổ sung nhánh xử lý ảnh đáy mắt [3].

Bảng 2.1. Ngưỡng chẩn đoán theo ADA

Xét nghiệm	Tiền đái tháo đường	Đái tháo đường
HbA1c (%)	5,7–6,4	$\geq 6{,}5$
Đường huyết lúc đói (mg/dL)	100–125	≥ 126
Nghiệm pháp 2 giờ (mg/dL)	140–199	≥ 200

2.2 Học máy trên dữ liệu bảng

Với tập $\{(\mathbf{x}_i, y_i)\}$ mất cân bằng, mô hình tối thiểu hóa entropy chéo có trọng số lớp, trong đó w_+ tăng ảnh hưởng của lớp dương:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[w_+\, y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]. \tag{2.1}$$

Rừng ngẫu nhiên giảm phương sai bằng trung bình nhiều cây [4]. XGBoost cộng dần các cây, mỗi cây khớp gradient của hàm mất mát với số hạng phạt độ phức tạp $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ [5].

Tác tử đọc xác suất cho người dùng nên xác suất phải đáng tin. Hiệu chỉnh isotonic tìm hàm đơn điệu g tối thiểu $\sum_i (y_i - g(\hat{p}_i))^2$ trên tập kiểm định [6]. Độ lệch được đo bằng sai số hiệu chỉnh kỳ vọng trên M khoảng [7]:

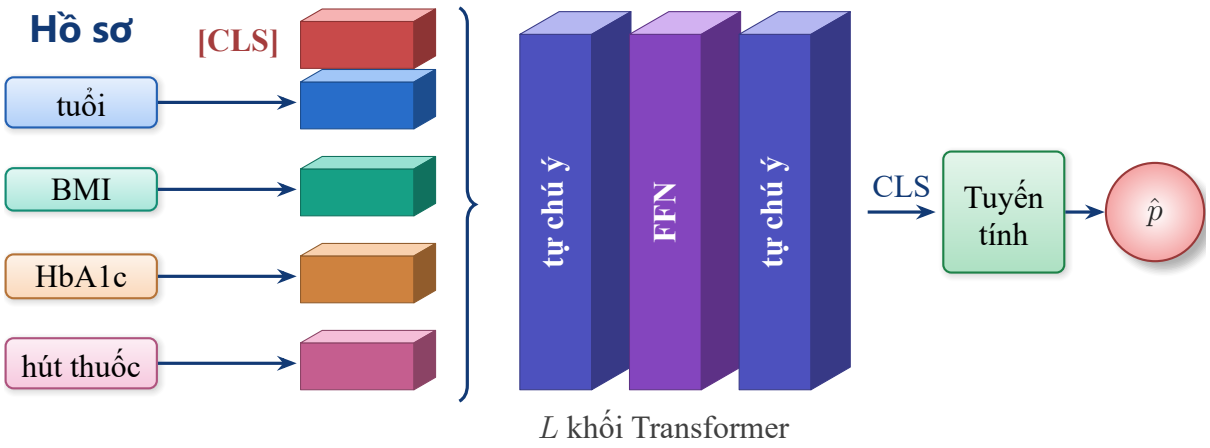
$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N}\left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|. \tag{2.2}$$

Giải thích từng dự đoán dùng SHAP: đầu ra được phân rã thành $f(\mathbf{x}) = \phi_0 + \sum_j \phi_j$, với ϕ_j là đóng góp Shapley của đặc trưng j [8]. Vì đích lệch lớp nên đồ án ưu tiên PR-AUC và F1 hơn accuracy; ngưỡng phân loại chỉ chọn trên tập kiểm định, tập kiểm tra khóa tới bước cuối.

2.3 Học sâu cho bảng và ảnh

FT-Transformer biến mỗi đặc trưng thành một token rồi cho các token tương tác qua tự chú ý [9], như Hình 2.1. Đặc trưng số x_j được nhúng thành $\mathbf{t}_j = x_j \mathbf{w}_j + \mathbf{b}_j$. Khối chú ý tính

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V. \tag{2.3}$$



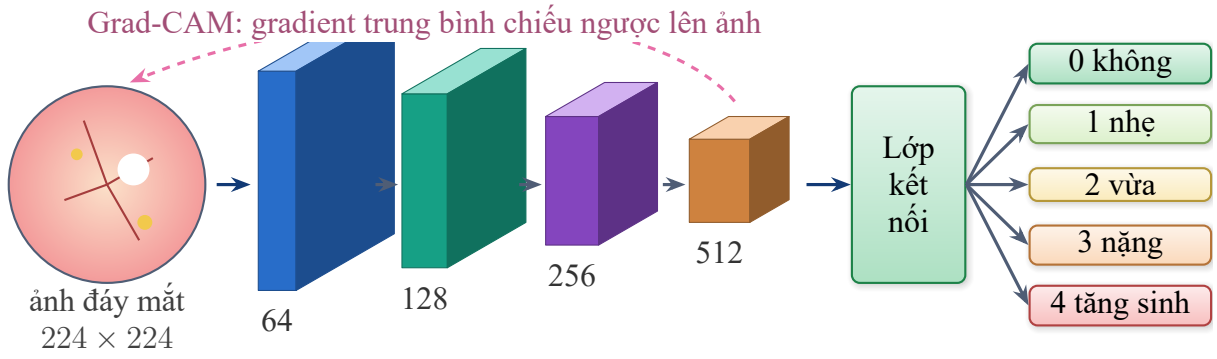
Hình 2.1. FT-Transformer: mỗi đặc trưng là một token, token CLS đưa ra dự đoán

Với ảnh, mạng tích chập trích đặc trưng phân cấp, kết nối tắt của ResNet giúp mạng sâu vẫn học được [10]. Grad-CAM chỉ ra vùng ảnh ảnh hưởng đến quyết định bằng trung bình gradient của từng kênh [11]:

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c = \text{ReLU}\Big(\sum_k \alpha_k^c A^k \Big). \tag{2.4}$$

Nhãn võng mạc có thứ tự nên đồ án dùng hệ số Kappa bậc hai, phạt nặng khi đoán lệch nhiều cấp:

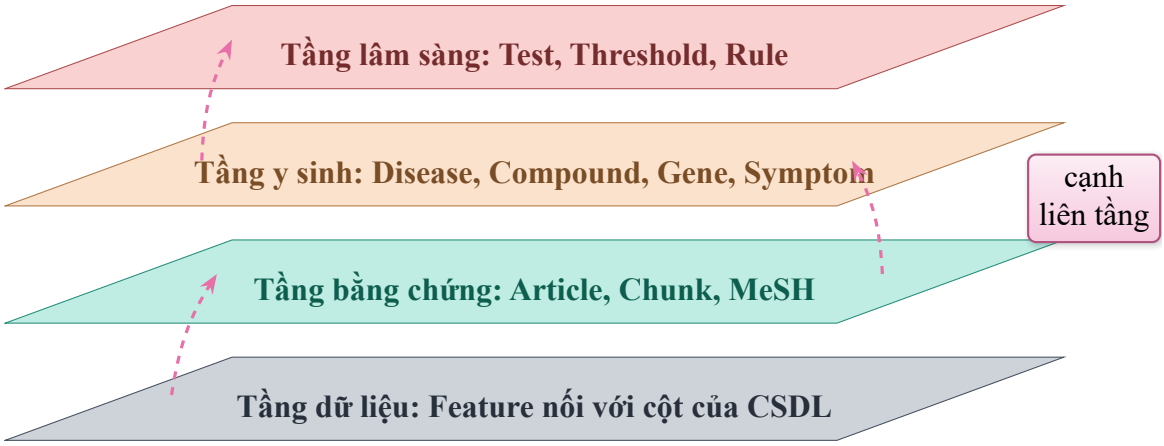
$$\kappa = 1 - \frac{\sum_{ij} w_{ij} O_{ij}}{\sum_{ij} w_{ij} E_{ij}}, \qquad w_{ij} = \frac{(i-j)^2}{(K-1)^2}. \tag{2.5}$$



Hình 2.2. Mạng tích chập phân cấp bệnh võng mạc và đường Grad-CAM

2.4 Đồ thị tri thức và RAG

Đồ thị tri thức biểu diễn sự kiện thành các bộ ba (chủ thể, quan hệ, đối tượng) trên đồ thị có hướng có nhãn [12]. DiaNexus tổ chức đồ thị thành bốn tầng nối với nhau bằng cạnh liên tầng (Hình 2.3), nhờ đó một đặc trưng trong CSDL đi được tới xét nghiệm, ngưỡng và câu bằng chứng.



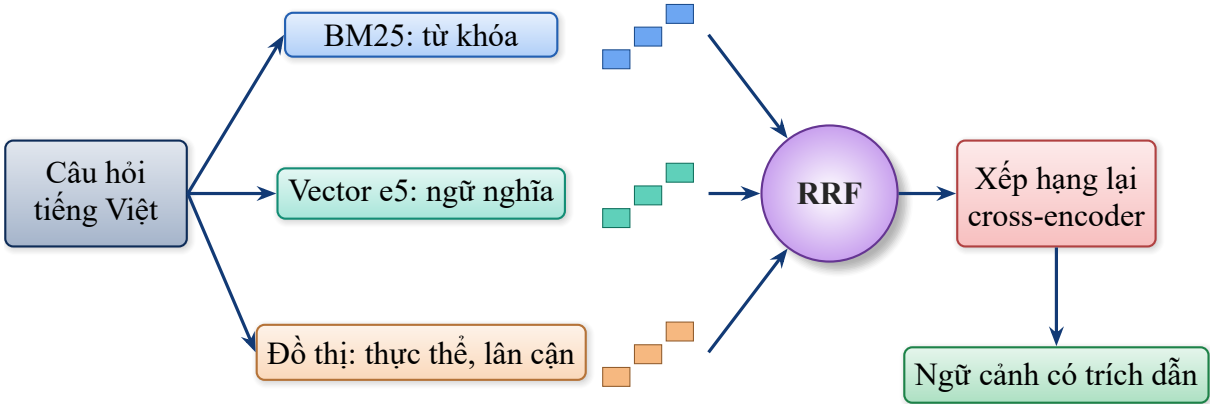
Hình 2.3. Bốn tầng của đồ thị tri thức

RAG đưa văn bản truy xuất vào ngữ cảnh của mô hình sinh để câu trả lời bám tài liệu [13]. BM25 chấm tài liệu d theo từ khóa [14]:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t,d)(k_1+1)}{f(t,d) + k_1\big(1 - b + b\,|d|/\overline{|d|}\big)}. \tag{2.6}$$

Truy xuất vector so độ tương tự cosin giữa hai embedding. Mỗi cách có điểm mù riêng, nên các danh sách được hợp lại bằng Reciprocal Rank Fusion [15], chỉ dùng thứ hạng và không cần chuẩn hóa điểm (Hình 2.4):

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}. \tag{2.7}$$



Hình 2.4. Truy xuất lai: ba bộ truy xuất, hợp RRF rồi xếp hạng lại

2.5 Hệ luật

Hệ luật suy diễn tiến lặp lại phép khớp: nếu điều kiện của luật thỏa mãn trên tập sự kiện thì thêm kết luận vào tập đó, cho tới khi không có gì mới. Luật không học từ dữ liệu nhưng có hai ưu điểm mà mô hình thống kê không có: mỗi kết luận truy ngược được về một điều khoản có nguồn, và hành vi là tất định. Vì vậy luật giữ các việc không được phép sai, như dấu hiệu khẩn cấp hay chống chỉ định thuốc.

2.6 Học tăng cường

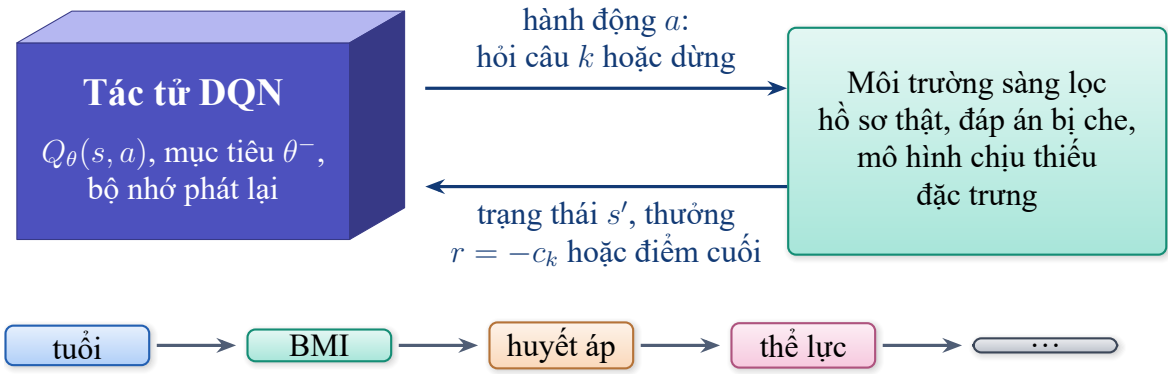
Bài toán ra quyết định tuần tự được mô hình hóa bằng MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ [16]. Q-learning cập nhật bảng giá trị theo phương trình Bellman [17]:

$$Q(s,a) \leftarrow Q(s,a) + \eta\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]. \tag{2.8}$$

Khi không gian trạng thái lớn, DQN xấp xỉ Q bằng mạng nơ-ron, dùng bộ nhớ phát lại và mạng đích [18]. Double DQN giảm ước lượng quá cao bằng cách tách chọn hành động khỏi đánh giá hành động [19]:

$$y = r + \gamma\, Q_{\theta^-}\big(s', \arg\max_{a'} Q_{\theta}(s',a')\big). \tag{2.9}$$

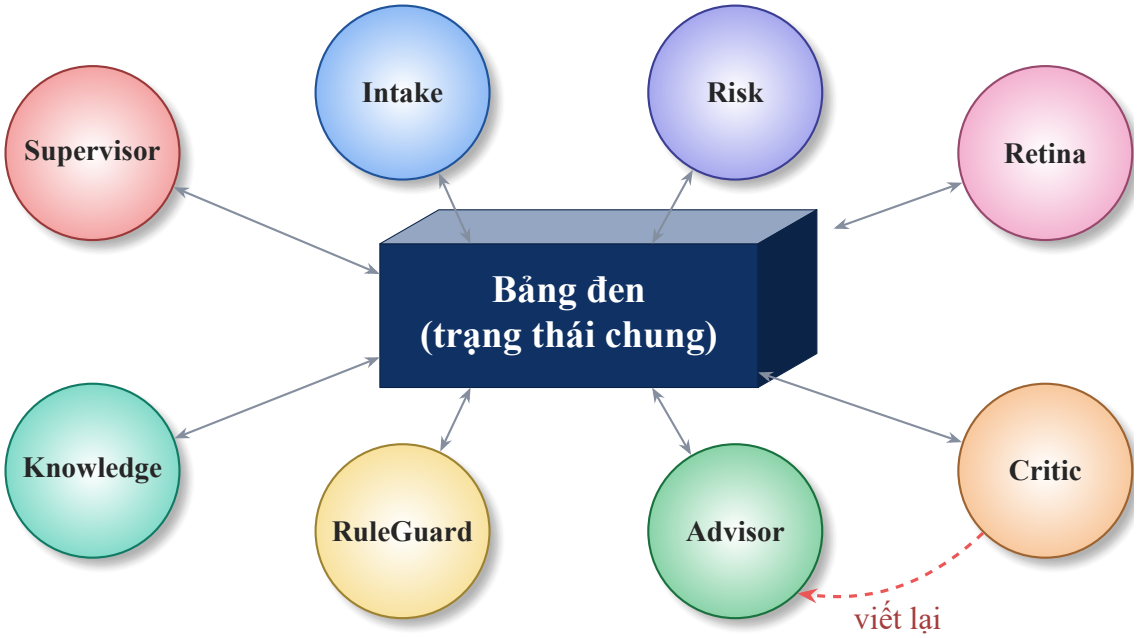
Chọn câu hỏi sàng lọc là dạng phân loại với đặc trưng có chi phí [20]: tác tử trả giá cho mỗi câu hỏi và nhận thưởng theo chất lượng kết luận cuối (Hình 2.5).



Hình 2.5. Sàng lọc thích ứng dưới dạng MDP

2.7 Tác tử và hệ đa tác tử

Tác tử cảm nhận môi trường và hành động để đạt mục tiêu [21]. Tác tử dùng LLM lặp vòng suy luận, gọi công cụ, quan sát [22]. Hệ đa tác tử chia việc cho các tác tử chuyên trách, phối hợp qua thông điệp hoặc một trạng thái chung [23]. Sự khác biệt với một đường ống tuần tự nằm ở chỗ luồng điều khiển được quyết định lúc chạy và có thể quay lui. Hình 2.6 là mô hình bảng đen của DiaNexus, các cạnh đỏ là vòng phản hồi.



Hình 2.6. Mô hình bảng đen và các vòng phản hồi của hệ đa tác tử

Chương 3
Dữ liệu và cơ sở dữ liệu

3.1 Các nguồn dữ liệu

DiaNexus dùng năm bộ dữ liệu bảng và một bộ ảnh đáy mắt. Mỗi bộ phục vụ một câu hỏi khác nhau, nên hệ thống không gộp chúng thành một bảng mà giữ nguyên ý nghĩa nhãn của từng nguồn. Bảng 3.1 tóm tắt quy mô sau khi làm sạch.

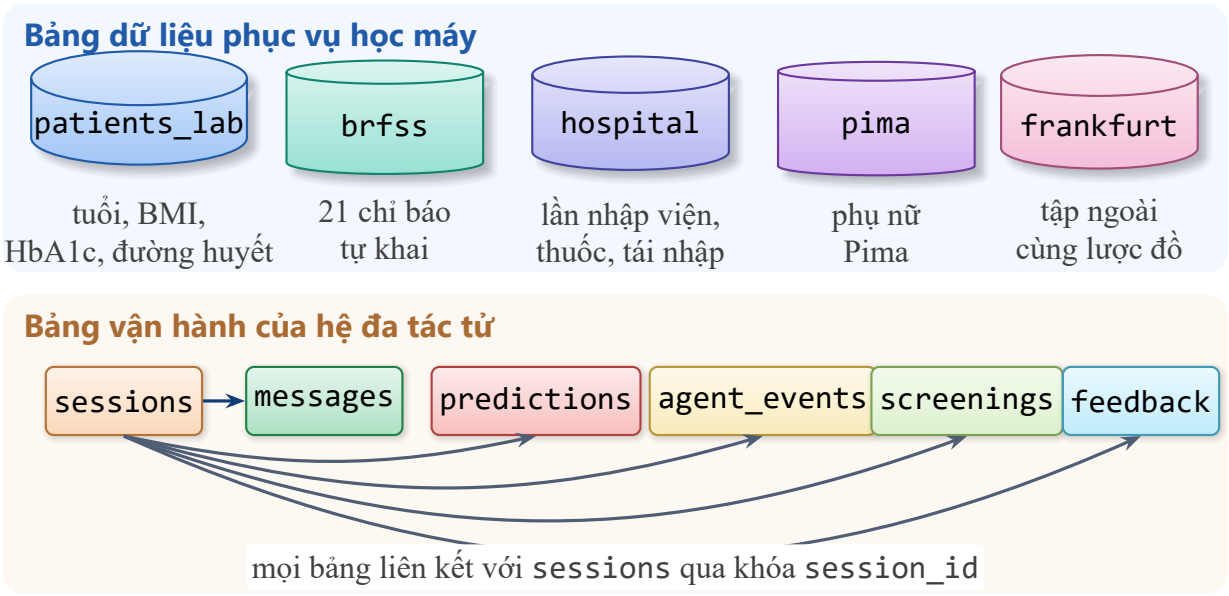
Bảng 3.1. Năm bộ dữ liệu bảng trong CSDL

Bảng	Dòng thô	Dòng sạch	Cột	Dương (%)	Bài toán
`patients_lab`	100.000	96.128	9	8,82	Chẩn đoán có xét nghiệm
`brfss`	253.680	229.474	22	15,29	Sàng lọc tự khai, RL
`hospital`	101.766	99.340	49	11,39	Tái nhập viện 30 ngày
`pima`	768	768	9	34,90	Đối chuẩn chéo miền
`frankfurt`	2.000	2.000	9	60,55	Đối chuẩn chéo miền

Bộ `brfss` là khảo sát hành vi sức khỏe của CDC, gồm các câu tự khai nên phù hợp cho sàng lọc khi người dùng chưa đi xét nghiệm. Bộ `hospital` ghi các lần nhập viện tại 130 bệnh viện Hoa Kỳ [24], bổ sung góc nhìn sau điều trị. Bộ Pima [25] và một tập ngoài 2.000 dòng cùng lược đồ (bảng `frankfurt`, được dùng lại từ bài tập trước, nguồn gốc chưa xác minh được) nhỏ nhưng có ích để thử mô hình trên quần thể khác với lúc huấn luyện.

3.2 Thiết kế CSDL

Toàn bộ dữ liệu nằm trong một tệp SQLite ở chế độ WAL, nên web và tác tử đọc ghi đồng thời được. Lược đồ chia hai nhóm như Hình 3.1: bảng dữ liệu chỉ đọc sau khi nạp, và bảng vận hành ghi lại mọi phiên làm việc. Nhờ bảng `agent_events`, mọi lượt chạy của hệ tác tử đều phát lại được trên website.



Hình 3.1. Lược đồ CSDL: bảng dữ liệu và bảng vận hành

Đoạn mã 3.1. Mở kết nối SQLite: bật WAL, khóa ngoại và bảo đảm đủ bảng vận hành

```
# dnx/db.py
def connect():
    """Kết nối mới, bật WAL và khóa ngoại, bảo đảm đủ bảng vận hành;
    ↪ hàng trả về dạng sqlite3.Row."""
    con = sqlite3.connect(DB_PATH, timeout=60, check_same_thread=False)
    con.row_factory = sqlite3.Row
    con.execute("PRAGMA journal_mode=WAL")
    con.execute("PRAGMA synchronous=NORMAL")
    con.execute("PRAGMA foreign_keys=ON")
    con.executescript(_DDL)
    return con
```

3.3 Làm sạch và chống rò rỉ

Làm sạch không đơn thuần là xóa dòng lỗi, mỗi quyết định đều thay đổi điều mô hình học được:

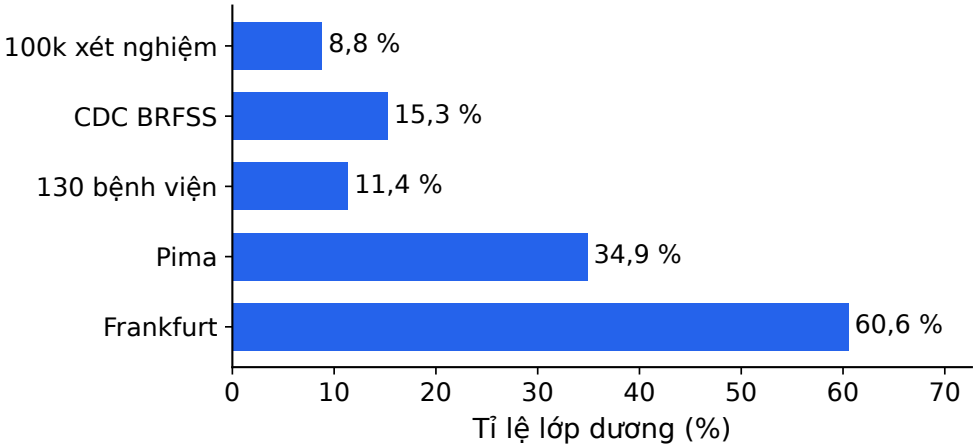
- `patients_lab` có 3.854 dòng trùng. Giữ lại thì một hồ sơ có thể nằm ở cả train và test, làm số đo đẹp giả tạo.
- `brfss` loại 24.206 dòng trùng, tỉ lệ dương tăng từ 13,93% lên 15,29%, tức các dòng trùng chủ yếu thuộc lớp âm.
- `hospital` loại 2.423 lần xuất viện đã tử vong hoặc chuyển chăm sóc cuối đời, vì những ca này không thể tái nhập viện. Cột `weight` thiếu tới 96,85% nên bị bỏ.

- `pima`, `frankfurt` dùng số 0 để ghi giá trị thiếu ở các cột sinh lý, ví dụ 374 dòng insulin của Pima. Các số 0 này được đổi thành giá trị thiếu.

Bẫy lớn nhất nằm ở `hospital`: 99.340 lần nhập viện chỉ thuộc về 69.987 bệnh nhân. Chia ngẫu nhiên theo dòng sẽ để cùng một người ở cả train và test, nên đồ án chia theo `patient_nbr`. Kiểm tra sau khi chia cho thấy số bệnh nhân trùng giữa các tập là 0.

3.4 Khám phá dữ liệu

Lớp dương ở cả năm bộ lệch nhau rất xa, từ 8,82% đến 60,55% (Hình 3.2). Vì vậy ngưỡng phân loại không dùng chung được giữa các nguồn, và Mục 4.4 sẽ đo chính hiệu ứng này.

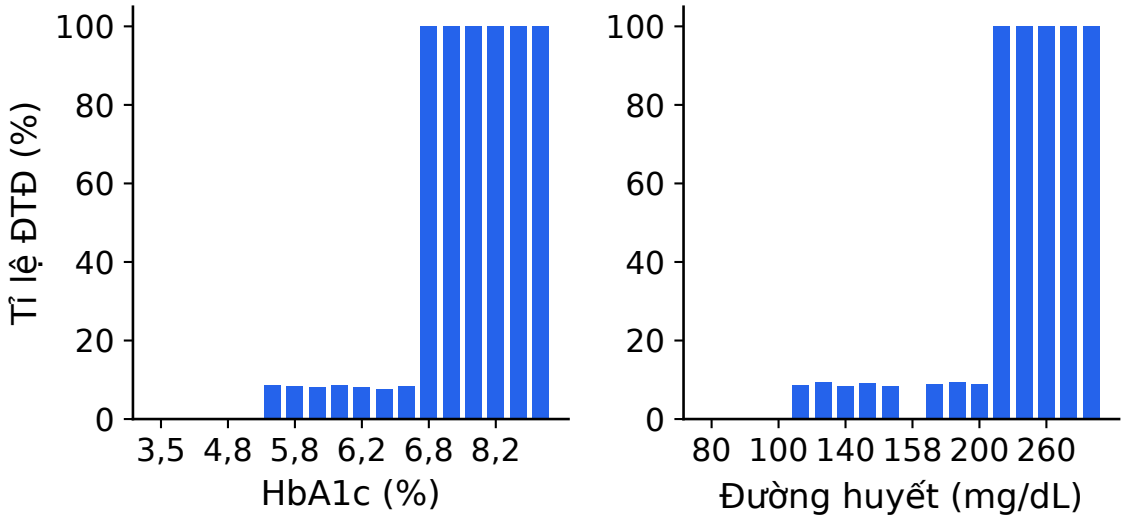


Hình 3.2. Tỉ lệ lớp dương của năm bộ dữ liệu

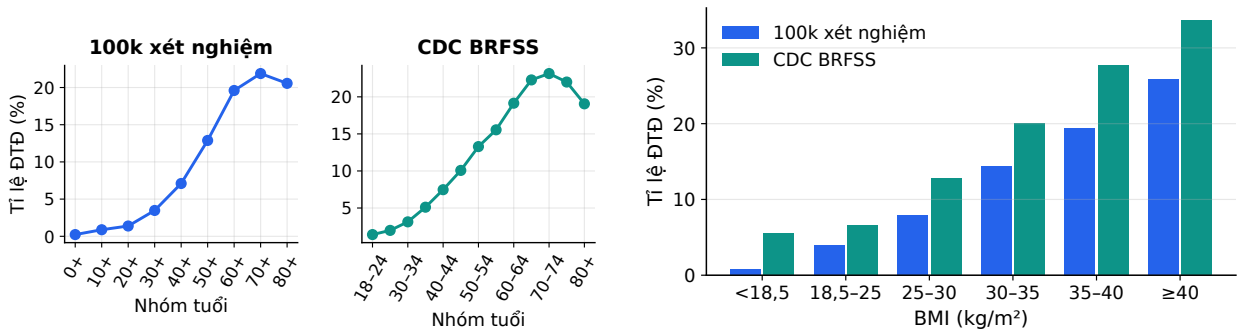
Hình 3.3 cho một phát hiện quan trọng về bộ `patients_lab`. Khi HbA1c từ 6,8 trở lên hoặc đường huyết từ 200 trở lên, tỉ lệ bệnh là đúng 100%; dưới hai mức này tỉ lệ chỉ quanh 8%. Nhãn của bộ này gần như sinh theo luật, nên điểm AUC rất cao ở chương sau phản ánh việc mô hình tìm lại ngưỡng, không phải năng lực dự đoán lâm sàng. Đây là giới hạn của dữ liệu và báo cáo ghi nhận rõ thay vì coi con số đó là thành tích.

Tuổi và BMI cho xu hướng nhất quán trên cả dữ liệu xét nghiệm lẫn khảo sát CDC (Hình 3.4). Với `brfss`, sức khỏe tự đánh giá và huyết áp cao tương quan mạnh nhất với bệnh (Hình 3.5), nghĩa là các câu hỏi đơn giản vẫn mang nhiều thông tin. Đây là cơ sở cho sàng lọc thích ứng ở Chương 6.

Với `hospital`, số lần nội trú năm trước là tín hiệu rõ nhất của tái nhập viện (Hình 3.6), còn chẩn đoán chính tạo chênh lệch rất ít. Điều này báo trước đây là bài toán khó.

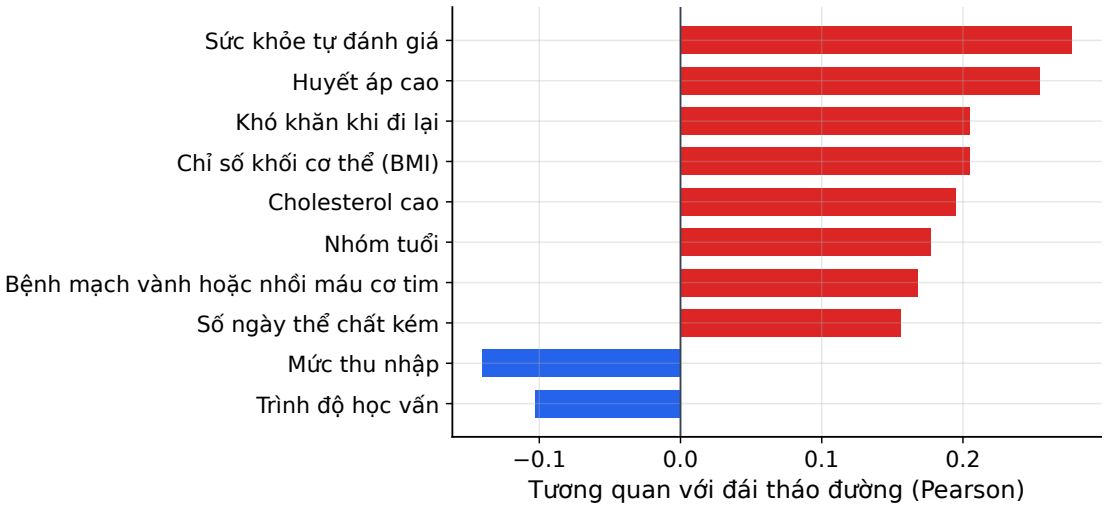


Hình 3.3. Tỉ lệ đái tháo đường theo từng mức HbA1c và đường huyết: bước nhảy tới 100%

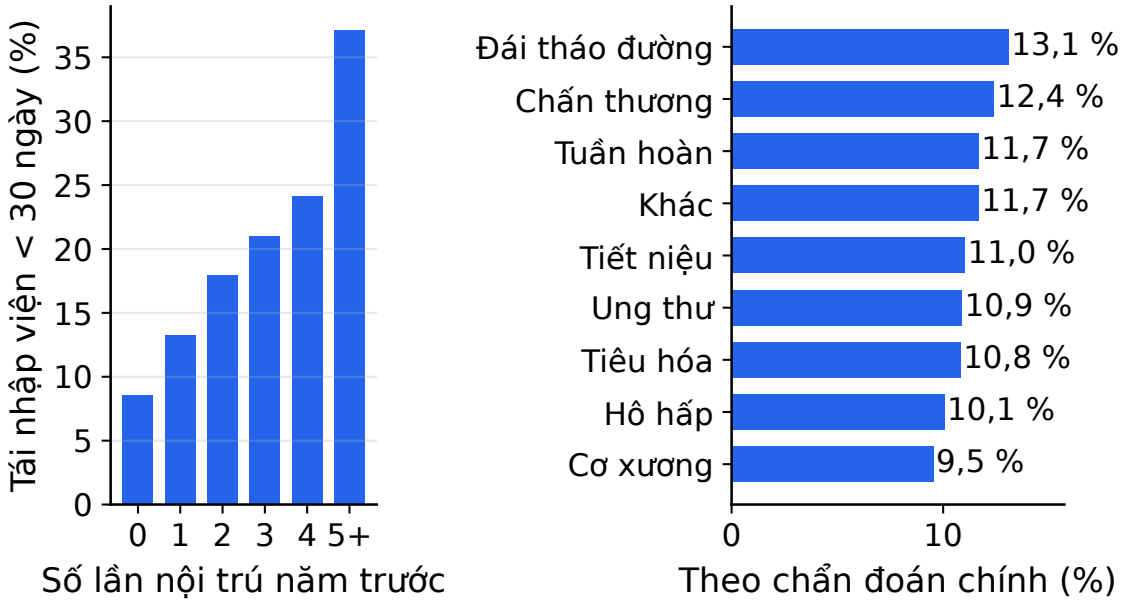


(a) Theo nhóm tuổi **(b)** Theo BMI

Hình 3.4. Tỉ lệ đái tháo đường tăng theo tuổi và BMI trên hai nguồn độc lập



Hình 3.5. Mười đặc trưng tương quan mạnh nhất với nhãn trong bảng `brfss`



Hình 3.6. Tỉ lệ tái nhập viện dưới 30 ngày theo số lần nội trú và chẩn đoán chính

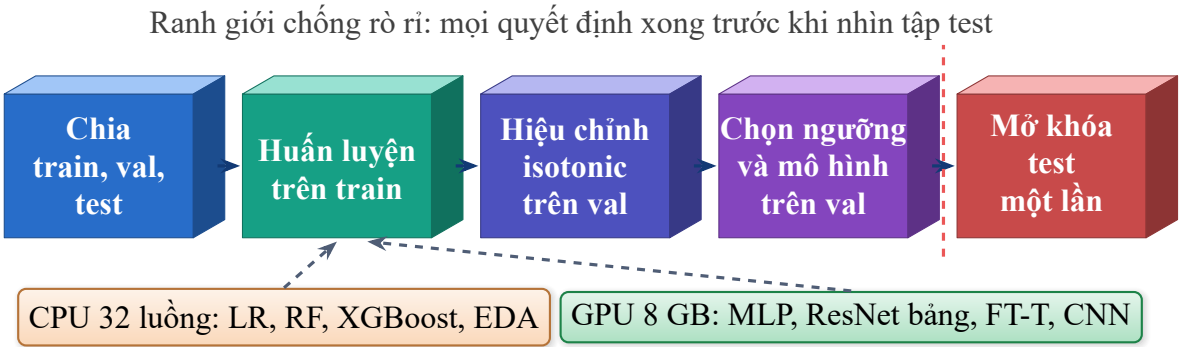
Bộ ảnh đáy mắt lấy từ youssefedweqd/Diabetic_Retinopathy_Detection (Hugging Face), cỡ 224 px, gồm 25.290 ảnh train, 2.810 ảnh val và 7.026 ảnh test, chia năm cấp bệnh và lệch mạnh về cấp 0.

Chương 4

Học máy, học sâu và dự đoán

4.1 Quy trình thí nghiệm

Ba bài toán bảng đi qua cùng một quy trình (Hình 4.1). Việc được chia cho hai loại phần cứng: mô hình cổ điển chạy trên CPU 32 luồng, mô hình sâu chạy trên GPU RTX 4060 và xếp hàng qua khóa GPU, nhờ đó hai nhóm chạy song song mà không tranh tài nguyên.



Hình 4.1. Quy trình chung: mọi lựa chọn diễn ra trên val, test chỉ mở một lần

4.2 Mô hình sâu cho dữ liệu bảng

Ba mô hình sâu dùng chung bước nhúng đặc trưng và vòng huấn luyện có dừng sớm. FT-Transformer dùng token CLS để gom thông tin từ mọi đặc trưng, đúng như mô tả ở Mục lý thuyết.

Đoạn mã 4.1. Cài đặt FT-Transformer bằng PyTorch

```python
# dnx/ml/deep.py
class FTTransformer(nn.Module):
    """Mỗi đặc trưng thành một token, thêm CLS, qua Transformer Encoder rồi phân loại trên CLS."""

    def __init__(self, n_num, cards, d=64, layers=3, heads=8, drop=0.1):
        super().__init__()
        self.w = nn.Parameter(torch.randn(n_num, d) * 0.1)
        self.b = nn.Parameter(torch.randn(n_num, d) * 0.1)
        self.emb = nn.ModuleList(nn.Embedding(c, d) for c in cards)
        self.cls = nn.Parameter(torch.randn(1, 1, d) * 0.1)
        layer = nn.TransformerEncoderLayer(d, heads, 2 * d, drop, "gelu", batch_first=True, norm_first=True)
        self.enc = nn.TransformerEncoder(layer, layers, enable_nested_tensor=False)
        self.head = nn.Sequential(nn.LayerNorm(d), nn.ReLU(), nn.Linear(d, 1))

    def forward(self, xn, xc):
        toks = [xn[:, :, None] * self.w + self.b]
        if len(self.emb):
            toks.append(torch.stack([e(xc[:, i]) for i, e in enumerate(self.emb)], 1))
        x = torch.cat([self.cls.expand(len(xn), -1, -1), *toks], 1)
        return self.head(self.enc(x)[:, 0])[:, 0]
```

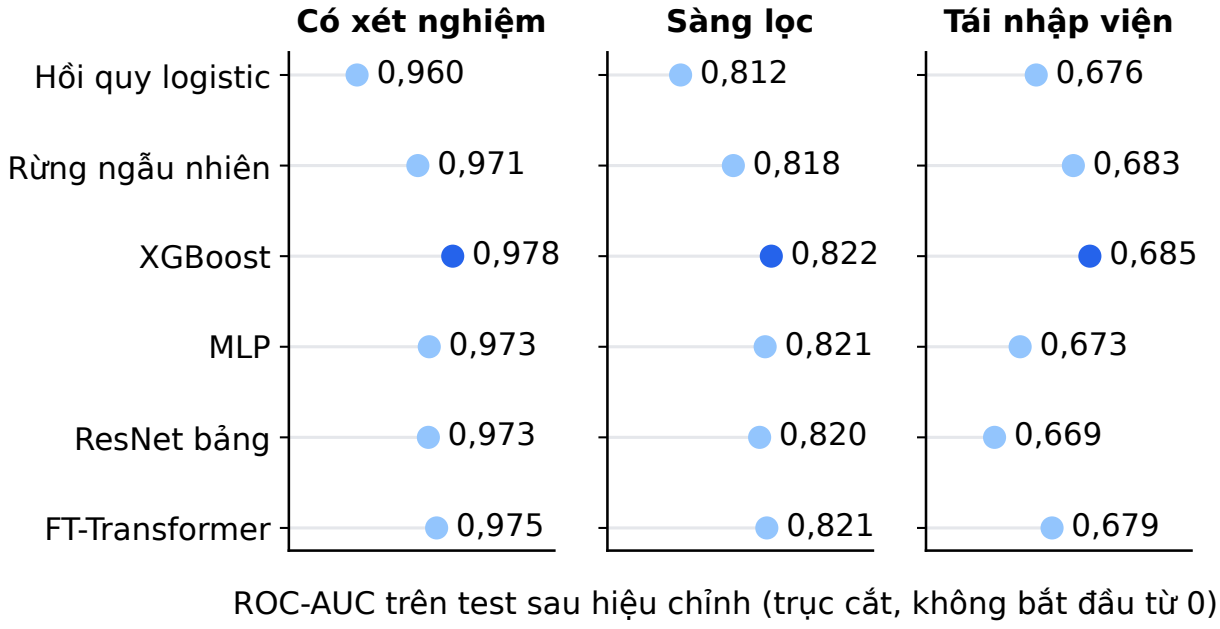
4.3 Kết quả trên tập kiểm tra

Bảng 4.1. Kết quả sáu mô hình trên test của ba bài toán sau hiệu chỉnh isotonic

Mô hình	Có xét nghiệm		Sàng lọc		Tái nhập viện	
	PR-AUC	F1	PR-AUC	F1	PR-AUC	F1
Hồi quy logistic	0,807	0,725	0,413	0,469	0,219	0,279
Rừng ngẫu nhiên	0,861	**0,811**	0,435	0,473	0,227	**0,299**
XGBoost	**0,874**	0,808	0,444	0,477	**0,237**	0,293
MLP	0,857	0,797	0,438	0,476	0,217	0,283
ResNet bảng	0,859	0,794	0,441	**0,478**	0,211	0,277
FT-Transformer	0,860	0,811	**0,444**	0,476	0,221	0,281

Bảng 4.1 và Hình 4.2 cho thấy mô hình sâu **không** vượt XGBoost trên dữ liệu bảng ở quy mô này. FT-Transformer tốn 95,05 giây trên bài có xét nghiệm, trong khi

XGBoost cần 9,013 giây và cho PR-AUC cao hơn. Ở bài sàng lọc, FT-Transformer nhỉnh nhất trên val nhưng khoảng cách chỉ $8{,}11 \times 10^{-5}$, nên hệ thống phục vụ XGBoost vì giải thích SHAP nhanh và chính xác trên cây.

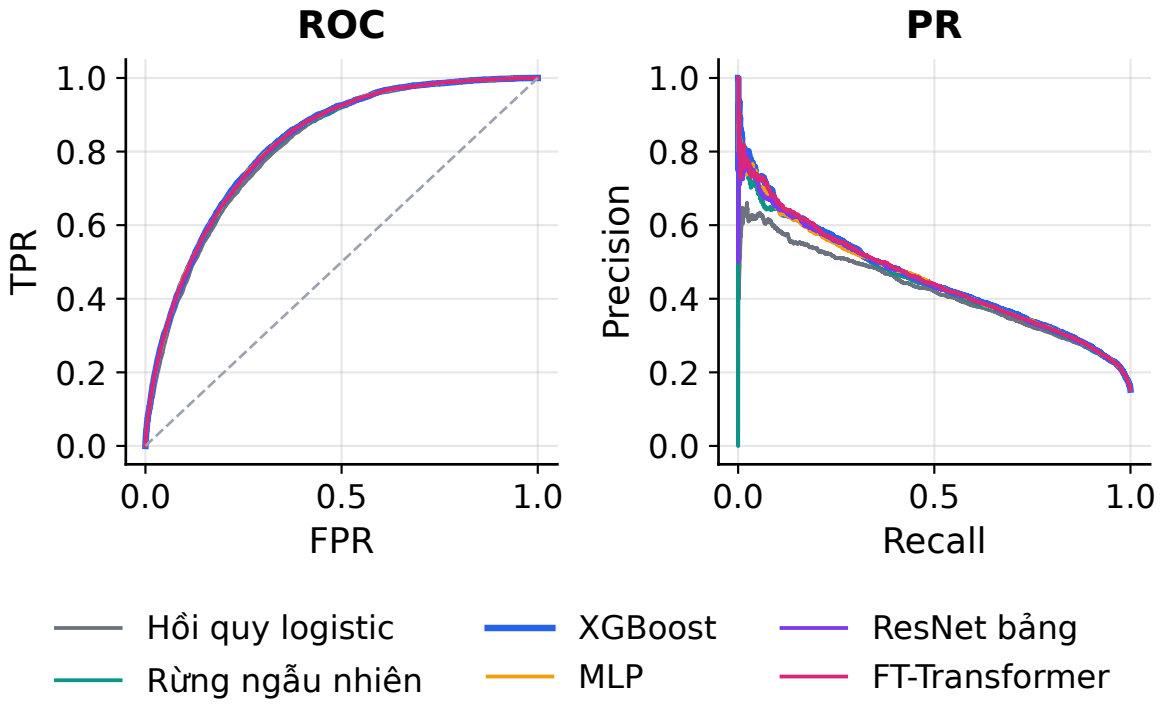


Hình 4.2. ROC-AUC của sáu mô hình trên ba bài toán

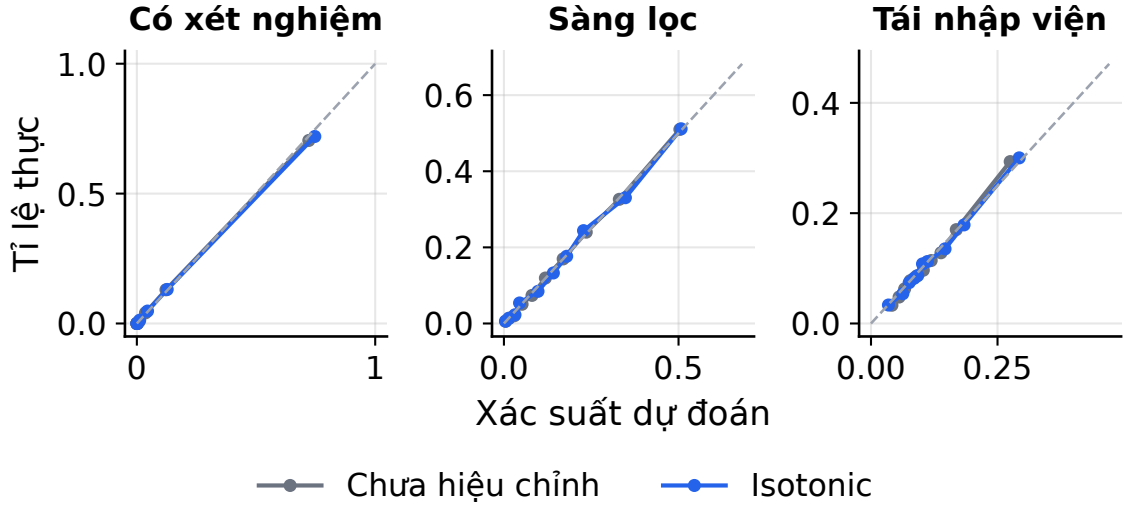
Mô hình phục vụ đạt ROC-AUC 0,9777 ở bài có xét nghiệm, với khoảng tin cậy bootstrap 0,9747–0,9806. Như đã nêu ở Chương 3, con số này cao vì nhãn gần như sinh theo ngưỡng. Hai bài còn lại phản ánh độ khó thật: sàng lọc tự khai đạt 0,8217, còn tái nhập viện chỉ đạt 0,6855. Tái nhập viện phụ thuộc nhiều vào yếu tố ngoài hồ sơ, nên hệ thống chỉ dùng kết quả này như tín hiệu tham khảo.

Kết quả hiệu chỉnh không như kỳ vọng ban đầu. Với XGBoost, isotonic chỉ giảm ECE ở bài tái nhập viện (0,0071 xuống 0,0061); ở bài có xét nghiệm ECE tăng từ 0,0036 lên 0,0050, ở bài sàng lọc từ 0,0046 lên 0,0071. XGBoost vốn đã khá chuẩn, nên isotonic không giúp thêm. Đồ án vẫn giữ bước này để mọi mô hình đi qua cùng quy trình, và cả hai mức sai số đều rất nhỏ, đường tin cậy ở Hình 4.4 bám sát đường chéo. Lớp phục vụ còn kẹp xác suất vào khoảng $[0{,}005;\ 0{,}995]$, vì hệ thống không được khẳng định chắc chắn một người mắc bệnh hay không.

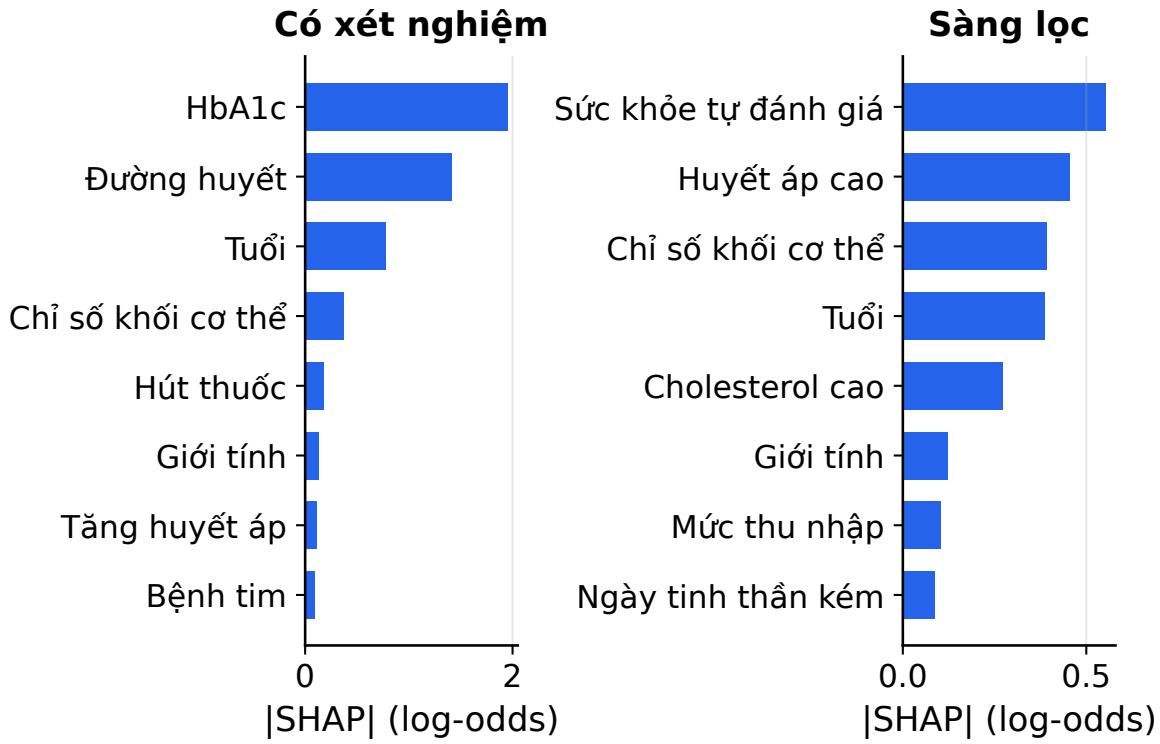
SHAP trong Hình 4.5 khớp với y văn: HbA1c, đường huyết đứng đầu khi có xét nghiệm; khi không có xét nghiệm, sức khỏe tự đánh giá, huyết áp cao và BMI thay thế vai trò đó.



Hình 4.3. ROC và Precision–Recall của bài sàng lọc không xét nghiệm



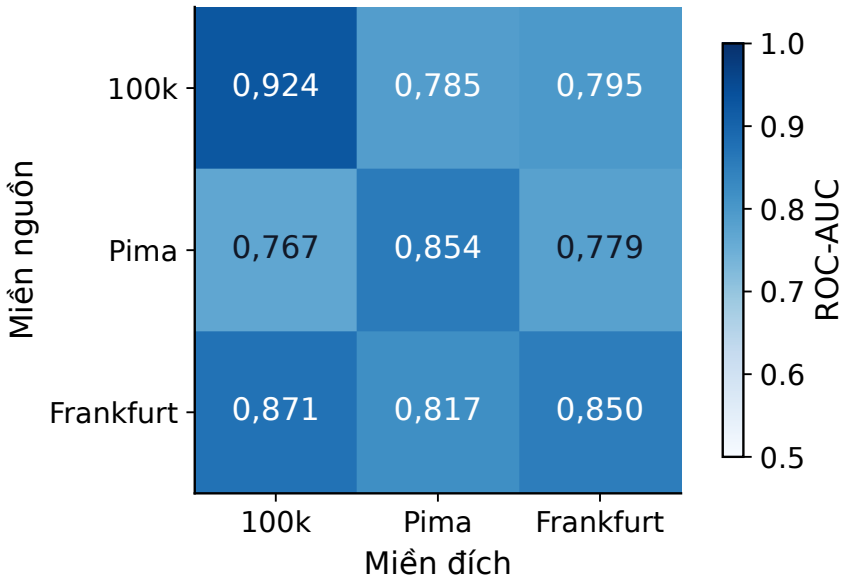
Hình 4.4. Biểu đồ tin cậy trước và sau hiệu chỉnh



Hình 4.5. Độ quan trọng SHAP trung bình của hai mô hình phục vụ

4.4 Đối chuẩn chéo miền

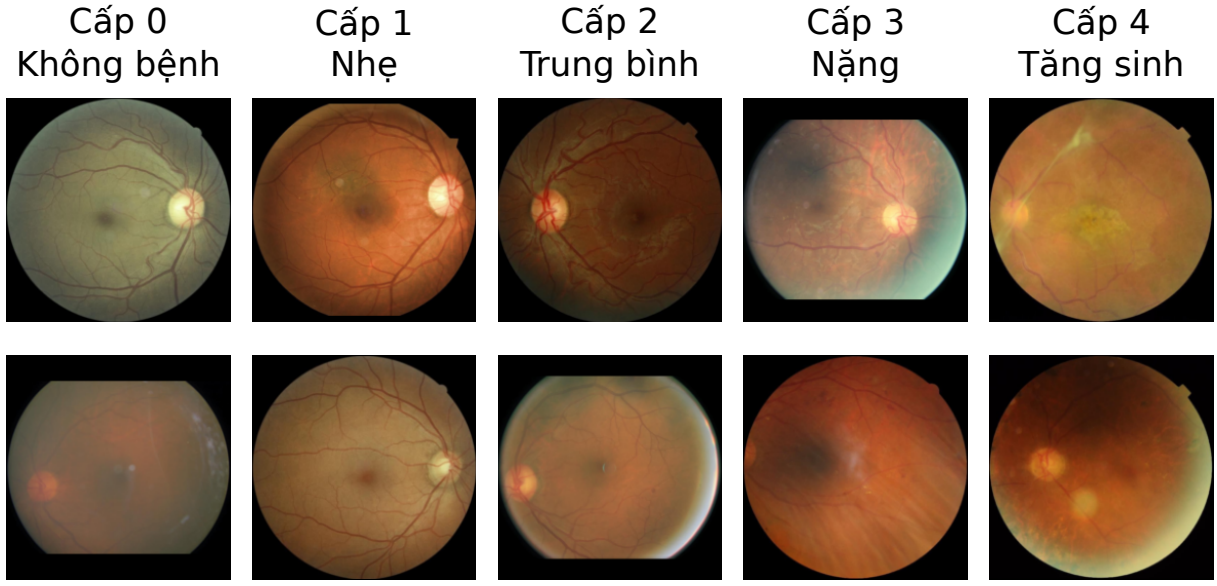
Để kiểm tra độ bền ngoài miền huấn luyện, đồ án huấn luyện trên một bộ rồi chấm trên bộ khác, chỉ bằng ba đặc trưng chung: tuổi, BMI, đường huyết. Hình 4.6 cho thấy ROC-AUC giảm vừa phải, nhưng thiệt hại thật nằm ở xác suất và ngưỡng: khi tỉ lệ bệnh của miền đích khác xa miền nguồn, sai số hiệu chỉnh tăng hàng chục lần, F1 tụt từ 0,58 xuống dưới 0,1. Một mô hình xếp hạng tốt vẫn có thể báo sai mức nguy cơ ở quần thể mới, đó là lý do DiaNexus nêu rõ nguồn dữ liệu của mô hình và không dùng xác suất như một chẩn đoán.



Hình 4.6. ROC-AUC của XGBoost khi huấn luyện ở miền nguồn, chấm ở miền đích

4.5 Phân cấp bệnh võng mạc

Nhánh ảnh dùng hai mạng tiền huấn luyện ImageNet là ResNet-34 và EfficientNet-B0 trên 25.290 ảnh. Lớp bệnh nặng rất hiếm (Hình 4.7), nên mỗi epoch lấy mẫu theo trọng số nghịch đảo căn tần suất. Tăng cường ảnh chạy thẳng trên GPU, không cần tiến trình nạp dữ liệu, nhờ đó mỗi mô hình chỉ cần khoảng 24 phút và tối đa 2.172 MB VRAM.



Hình 4.7. Ảnh đáy mắt thật của năm cấp bệnh

Theo Bảng 4.2, EfficientNet-B0 thắng trên val với khoảng cách rất nhỏ nên được chọn, dù trên test ResNet-34 nhỉnh hơn; đồ án giữ đúng quy tắc chọn trên val.

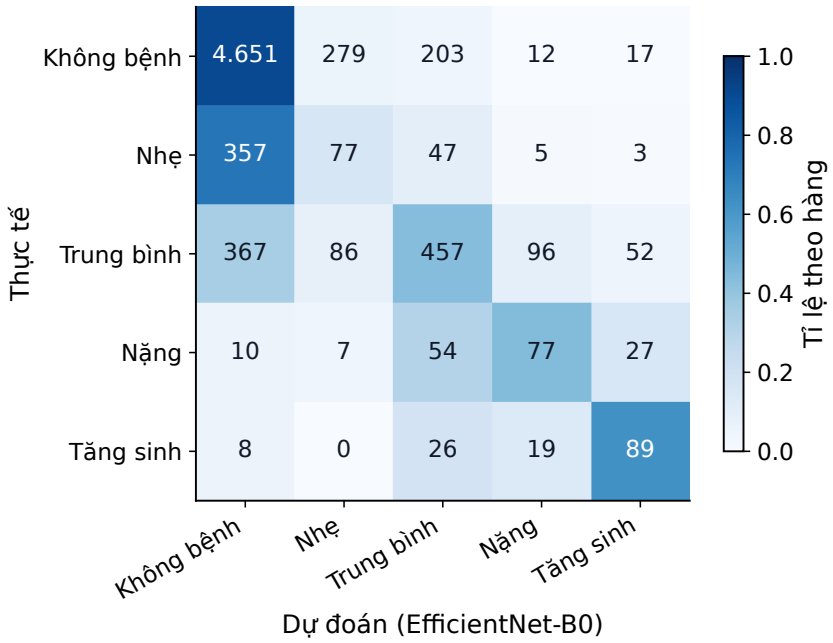
Đoạn mã 4.2. Tăng cường ảnh trên GPU: lật, xoay, phóng, sáng và tương phản

```python
# dnx/ml/retina.py
def augment(x):
    """Lật, xoay ±25°, phóng ±10 %, sáng/tương phản theo từng ảnh, chạy trên GPU nên không cần worker nạp dữ liệu."""
    import torch
    import torch.nn.functional as F
    n, dev = len(x), x.device
    x = x.permute(0, 3, 1, 2).float() / 255
    a = (torch.rand(n, device=dev) - 0.5) * 2 * np.deg2rad(25)
    z = 1 + (torch.rand(n, device=dev) - 0.5) * 0.2
    sx = torch.where(torch.rand(n, device=dev) < 0.5, -1.0, 1.0)
    sy = torch.where(torch.rand(n, device=dev) < 0.5, -1.0, 1.0)
    cos, sin = torch.cos(a) * z, torch.sin(a) * z
    zero = torch.zeros(n, device=dev)
    th = torch.stack([torch.stack([cos * sx, -sin, zero], 1),
     ↪  torch.stack([sin, cos * sy, zero], 1)], 1)
    x = F.grid_sample(x, F.affine_grid(th, x.shape,
     ↪  align_corners=False), align_corners=False,
     ↪  padding_mode="zeros")
    x = x * (0.85 + 0.3 * torch.rand(n, 1, 1, 1, device=dev))
    mean = x.mean((2, 3), keepdim=True)
    x = (x - mean) * (0.85 + 0.3 * torch.rand(n, 1, 1, 1, device=dev))
     ↪  + mean
    m = torch.tensor(MEAN, device=dev).view(1, 3, 1, 1)
    s = torch.tensor(STD, device=dev).view(1, 3, 1, 1)
    return (x.clamp(0, 1) - m) / s
```

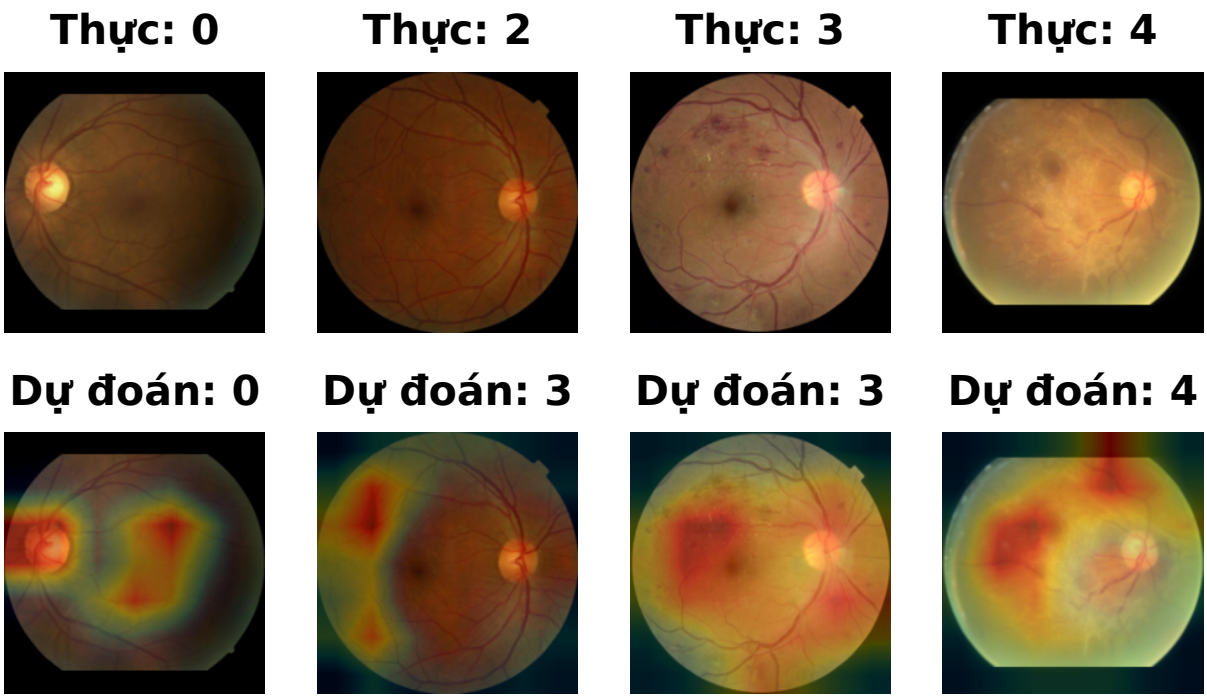
Bảng 4.2. Kết quả phân cấp võng mạc trên test (chọn mô hình bằng QWK trên val)

Mô hình	Accuracy	Macro-F1	QWK val	QWK test
ResNet-34	0,7589	0,5209	0,6873	0,6885
EfficientNet-B0	0,7616	0,4963	0,6886	0,6860

QWK khoảng 0,69 cho thấy mô hình bắt được thứ tự nặng nhẹ, nhưng Hình 4.8 chỉ rõ điểm yếu: cấp 1 có F1 chỉ 0,1642, vì tổn thương sớm rất nhỏ ở độ phân giải 224 px. Bộ dữ liệu cũng không có mã bệnh nhân, nên không loại trừ được hai mắt của một người rơi vào hai tập khác nhau. Kết quả chỉ nên xem như gợi ý đi khám mắt.



Hình 4.8. Ma trận nhầm lẫn của EfficientNet-B0 trên test



Hình 4.9. Grad-CAM: vùng ảnh mà mô hình dựa vào để phân cấp

4.6 Phục vụ mô hình

Lớp phục vụ chọn chế độ theo dữ liệu người dùng cung cấp: có HbA1c hoặc đường huyết thì dùng mô hình xét nghiệm, nếu không thì dùng mô hình sàng lọc. Kết quả gồm xác suất đã hiệu chỉnh, ngưỡng và danh sách đóng góp SHAP, sẵn sàng cho tác tử Risk và trang what-if.

Đoạn mã 4.3. Hàm dự đoán dùng chung cho tác tử và website

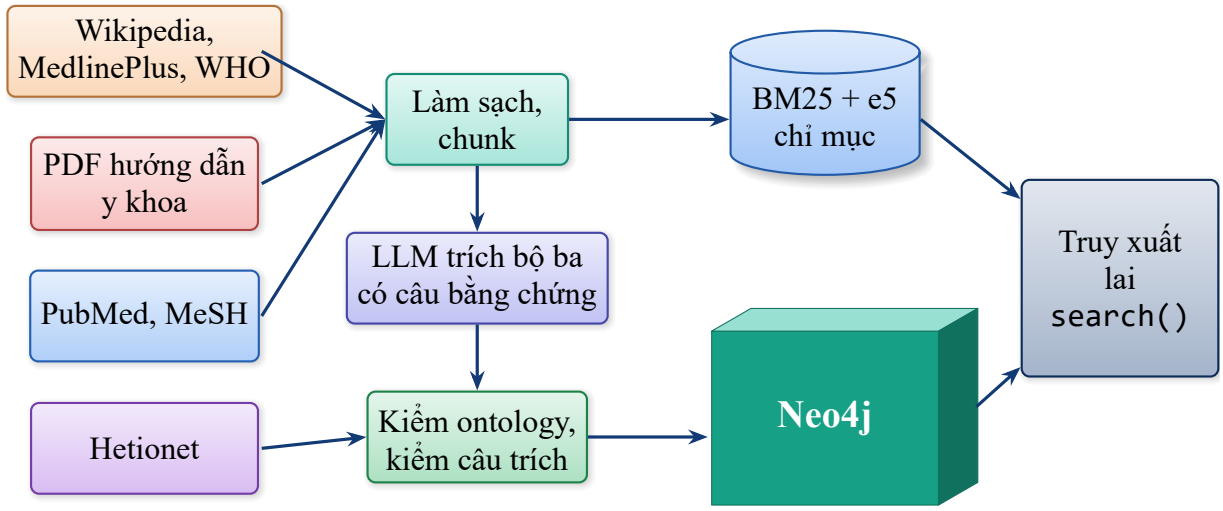
```python
# dnx/ml/serve.py
def predict(profile, mode="auto"):
    """Xác suất đã hiệu chỉnh và đóng góp SHAP (thang log-odds, gộp cột one-hot) giảm dần theo trị tuyệt đối."""
    task = _pick_mode(profile, mode)
    b = _bundle(task)
    row = (_row_full if task == "full" else _row_screening)(profile)
    raw, prob, dm = _score(task, _transform(task, row))
    contribs = b["booster"].predict(dm, pred_contribs=True)[0]
    by = {}
    for g, c in zip(b["groups"], contribs[:-1]):
        by[g] = by.get(g, 0.0) + float(c)
    shap = [{"feature": f, "label_vi": LABEL_VI[f], "value": row.get(f), "contribution": c}
            for f, c in sorted(by.items(), key=lambda kv: -abs(kv[1]))]
    th = b["meta"]["threshold"]
    return {"mode": task, "prob": prob, "label": int(prob >= th), "threshold": th, "model": b["meta"]["model"],
            "calibrated": True, "shap": shap, "raw_prob": raw, "base_value": float(contribs[-1]),
            "imputed": [c for c in TASKS[task]["num"] + TASKS[task]["cat"] if _na(row.get(c))]}
```

Chương 5

Cơ sở tri thức, Neo4j và RAG

5.1 Thu thập và tiền xử lý

Cơ sở tri thức của DiaNexus đến từ bốn loại nguồn: bài phổ biến kiến thức (Wikipedia tiếng Việt, MedlinePlus, NIDDK, WHO, CDC), tài liệu PDF hướng dẫn chuyên môn (trong đó có hướng dẫn của Bộ Y tế), tóm tắt bài báo khoa học kèm từ khóa MeSH qua Europe PMC, và đồ thị y sinh Hetionet [26]. Hình 5.1 là toàn bộ đường đi của dữ liệu.



Hình 5.1. Đường ống từ tài liệu thô đến chỉ mục và đồ thị

Mọi nguồn được ghi lại URL và ngày lấy. Văn bản được chia theo câu thành đoạn tối đa 180 từ, chồng lấn tối đa 30 từ, tổng cộng 51.164 đoạn từ 23.064 tài liệu (Bảng 5.1). Kho rất lệch: tóm tắt bài báo chiếm phần lớn, tiếng Việt chỉ có 3.358 đoạn, nên việc truy xuất chéo ngôn ngữ là yêu cầu bắt buộc.

Ngưỡng chẩn đoán được trích riêng thành dữ liệu có cấu trúc: 22 ngưỡng, mỗi ngưỡng gắn với đoạn văn làm bằng chứng. Hệ luật ở Chương 6 đọc đúng các ngưỡng này.

Bảng 5.1. Nguồn của cơ sở tri thức

Nguồn	Tài liệu	Đoạn	Ngôn ngữ
Europe PMC	22.518	43.690	Anh
Wikipedia	196	3.173	Việt
MedlinePlus	222	2.296	Anh
NIDDK	67	1.122	Anh
CDC	56	391	Anh
WHO	14	307	Anh
Tờ thông tin dịch	12	101	Việt
Bộ Y tế	2	84	Việt

5.2 Ontology và đồ thị trên Neo4j

Ontology được khai báo đóng trong mã nguồn. Đồ thị dùng 24 nhãn nút và 33 loại quan hệ. Mỗi quan hệ do LLM trích có ràng buộc miền và trị, nên cạnh sai loại bị loại ngay bằng mã thay vì nhờ LLM tự kiểm.

Đoạn mã 5.1. Ontology đóng: mỗi quan hệ quy định loại chủ thể và đối tượng hợp lệ

```
# dnx/kb/ontology.py
LLM_RELATIONS = {
    "TREATS": (("Compound", "Lifestyle"), _CLIN),
    "PREVENTS": (("Compound", "Lifestyle"), _CLIN),
    "CAUSES": (("Disease", "Complication", "Compound", "RiskFactor"),
     ↪  ("Disease", "Complication", "Symptom")),
    "RISK_FACTOR_FOR": (("RiskFactor", "Lifestyle", "Disease",
     ↪  "Complication"), _CLIN),
    "COMPLICATION_OF": (("Complication", "Disease"), _CLIN),
    "PRESENTS": (_CLIN, ("Symptom",)),
    "DIAGNOSED_BY": (_CLIN, ("Test",)),
    "MONITORED_BY": (_CLIN + ("Compound",), ("Test",)),
    "AFFECTS": (_CLIN + ("Compound",), ("Anatomy",)),
    "ASSOCIATED_WITH": (_CLIN + ("Gene", "RiskFactor"), _CLIN +
     ↪  ("Gene", "RiskFactor")),
    "CONTRAINDICATED_IN": (("Compound",), _CLIN),
}
```

Đồ thị đang chạy trên Neo4j có 108.749 nút và 902.689 cạnh, ghép từ bốn lớp:

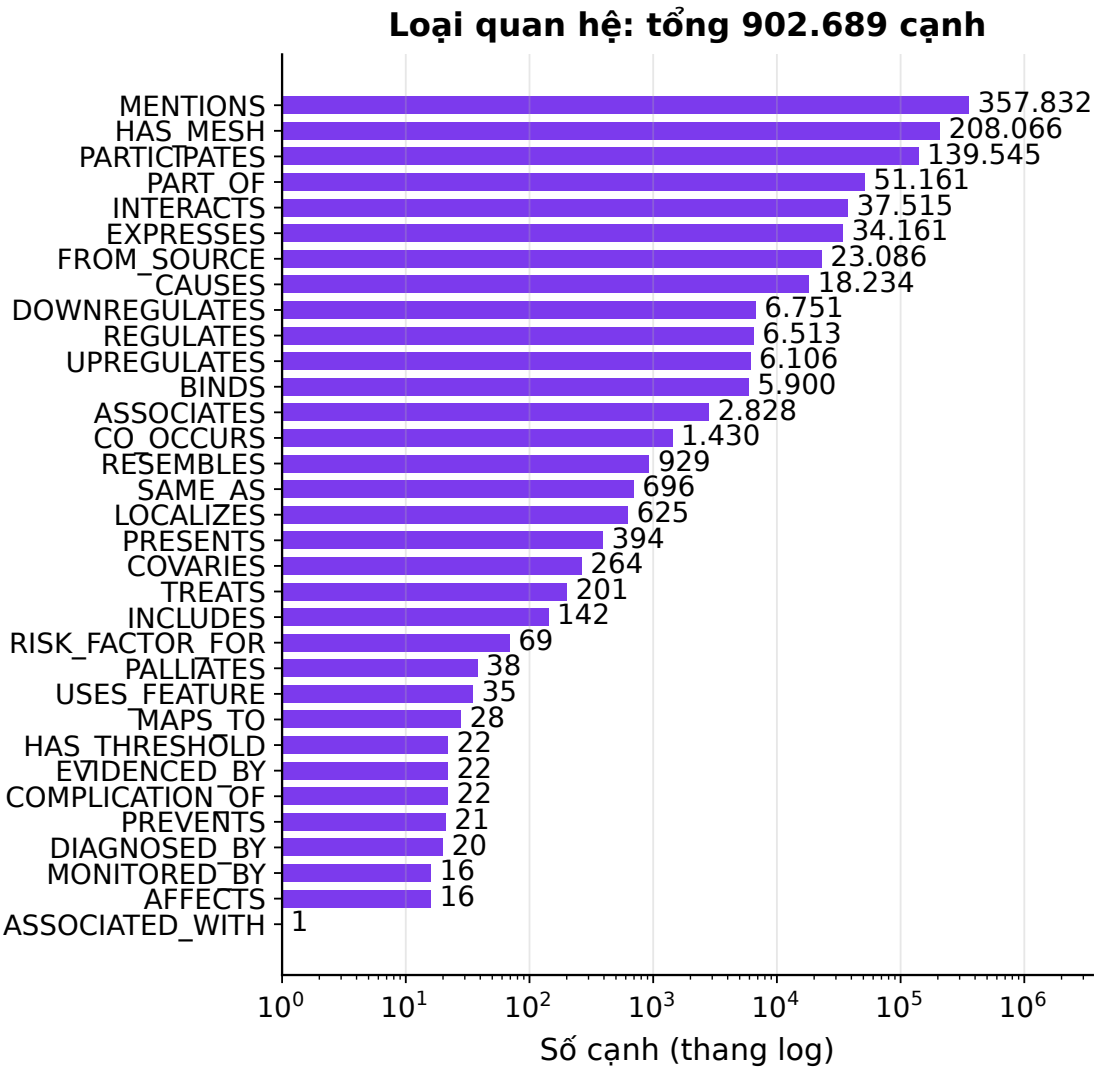
- **Hetionet:** từ 11 bệnh gốc, giữ 25.754 nút và 260.000 cạnh trong tổng số 2.250.197 cạnh của bản đầy đủ, vì máy chỉ cấp 1 GB heap cho Neo4j.
- **MeSH:** từ khóa của bài báo tạo nút chủ đề và quan hệ đồng xuất hiện.

- **Văn bản:** tài liệu, đoạn văn và quan hệ `MENTIONS` nối đoạn văn với thực thể.
- **Trích bằng LLM:** Qwen đọc 404 đoạn, sinh 667 cạnh thô. Bộ lọc mã nguồn loại 196 cạnh có câu bằng chứng không nằm trong đoạn văn và 87 cạnh sai miền, trị, còn lại 313 cạnh.

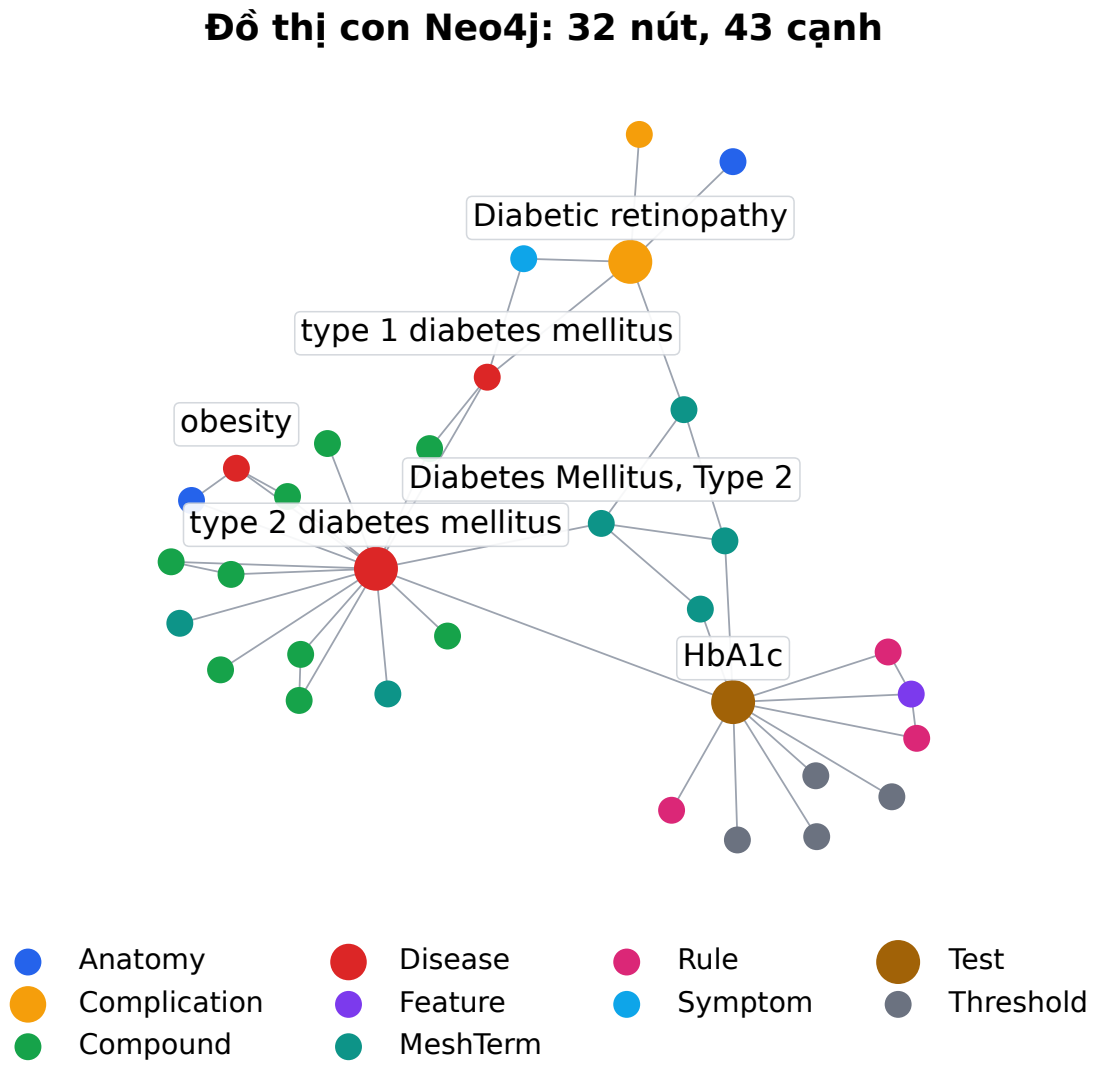
Đồ thị không chỉ để trình diễn mà trả lời được câu hỏi nhiều bước. Truy vấn dưới đây đi từ đặc trưng của mô hình học máy tới ngưỡng rồi tới đoạn văn làm bằng chứng, tức là nối hai nhánh của sơ đồ đề bài:

```
MATCH (f:Feature {name:'hba1c'})-[:MAPS_TO]->(t:Test)
      -[:HAS_THRESHOLD]->(h)-[:EVIDENCED_BY]->(c:Chunk)
RETURN f.name_vi, h.op, h.value, h.unit, h.meaning, c.source
```

Bộ lọc chỉ kiểm được hình thức, chưa kiểm được ý nghĩa. Lấy mẫu ngẫu nhiên 50 cạnh để chấm chặt, chỉ 27 cạnh đúng, tức precision 0,5400, khoảng Wilson 95% từ 0,4040 đến 0,6703. Người chấm là một tác tử AI, không phải bác sĩ. Vì vậy cạnh có `origin='llm'` chỉ dùng để gợi ý, không làm bằng chứng y khoa, và hệ thống không dùng chúng cho ngưỡng hay luật.



Hình 5.2. Số cạnh theo từng loại quan hệ trong đồ thị (thang log)



Hình 5.3. Một phần đồ thị thật quanh đái tháo đường type 2

5.3 Truy xuất lai

Hàm `search` kết hợp ba bộ truy xuất. BM25 bắt tên thuốc và mã xét nghiệm chính xác. Vector e5 đa ngôn ngữ [27] nối câu hỏi tiếng Việt với tóm tắt tiếng Anh. Bộ truy xuất đồ thị nhận diện thực thể trong câu hỏi, đi tới lân cận trên Neo4j rồi lấy các đoạn văn nhắc tới chúng, xếp bằng trọng số hiếm kết hợp độ tương tự cosin. Ba danh sách được hợp bằng RRF; đoạn nào có nhiều bộ cùng tìm thấy sẽ được đẩy lên.

Đoạn mã 5.2. Bộ truy xuất đồ thị: từ thực thể tới lân cận tới đoạn văn

```python
# dnx/kb/search.py
def _graph(query, vec, depth):
    """Thực thể trong câu hỏi -> lân cận trên Neo4j -> chunk nhắc tới chúng; xếp bằng cosin + thưởng thực thể hiếm."""
    seeds = [h[0] for h in _link(query)]
    if not seeds:
        return []
    near = store.cypher(NEIGHBOR_Q, ids=seeds, clinical=CLINICAL_RELS, n=NEIGHBORS)
    m, eidx = store.mention_index()
    weights = {e: W_SEED for e in seeds}
    for r in near:
        weights.setdefault(r["nid"], W_NEIGHBOR)
    n_docs = m.shape[0]
    s = np.zeros(n_docs, dtype=np.float32)
    for e, w in weights.items():
        j = eidx.get(e)
        if j is None:
            continue
        col = m.indices[m.indptr[j]:m.indptr[j + 1]]
        if len(col):
            s[col] += w * float(np.log1p(n_docs / len(col)))
    idx = np.flatnonzero(s)
    if not len(idx):
        return []
    # thực thể chung (hub) khớp hàng chục nghìn chunk nên không cắt theo điểm thực thể; cosin quyết định, thực thể cộng điểm thưởng
    final = store.vectors()[idx] @ vec + W_ENTITY * s[idx] / s[idx].max()
    order = np.argsort(-final, kind="stable")[:depth]
    return [(int(idx[i]), float(final[i])) for i in order]
```

Nếu Neo4j không chạy, chế độ lai ghi cảnh báo rồi tiếp tục với hai bộ còn lại, và kết quả không có nhãn `graph`, nên người đọc biết nguồn nào đã tham gia.

Đoạn mã 5.3. Hợp các danh sách bằng Reciprocal Rank Fusion

```python
# dnx/kb/search.py
def _rrf(lists, k):
    fused, who = {}, {}
    for name, rows in lists.items():
        for rank, (i, _) in enumerate(rows, 1):
            fused[i] = fused.get(i, 0.0) + 1.0 / (RRF_C + rank)
            who.setdefault(i, []).append(name)
    top = sorted(fused.items(), key=lambda x: -x[1])[:k]
    return [(i, s, who[i]) for i, s in top]
```

5.4 Đánh giá truy xuất

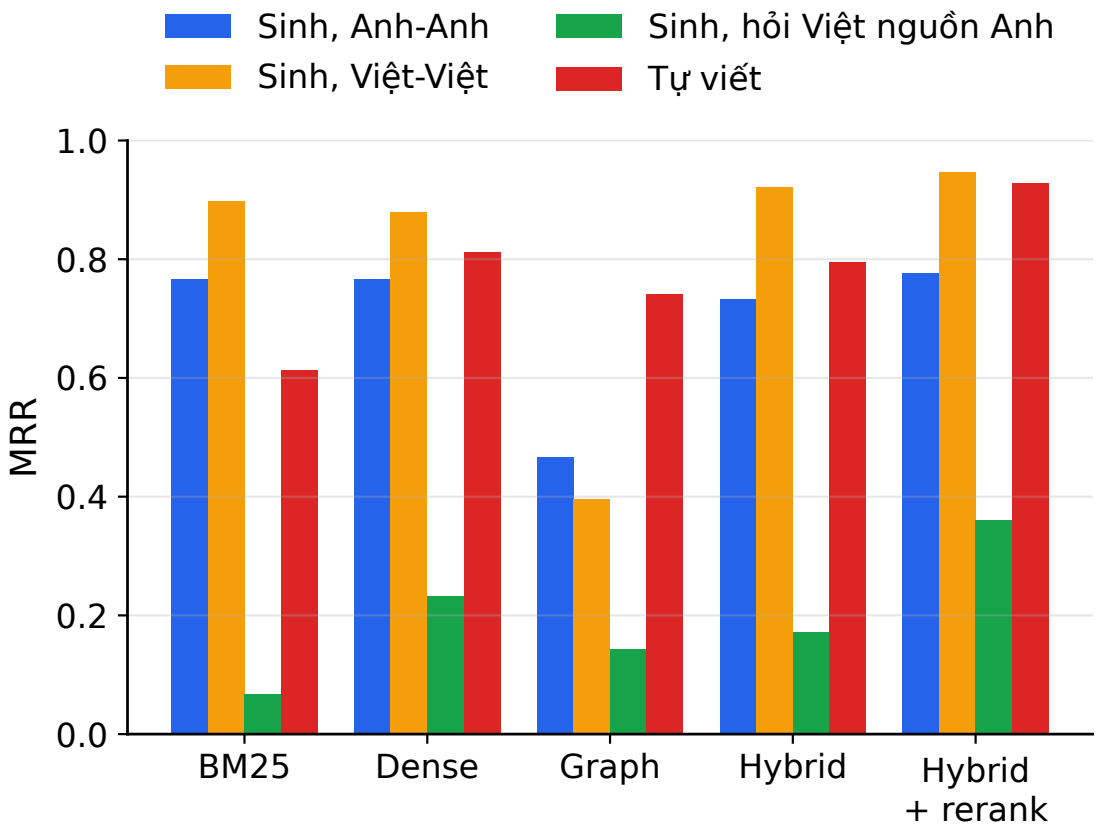
Bộ đánh giá gồm 174 câu hỏi: 42 câu tự viết và 132 câu do LLM sinh từ chính đoạn văn, trong đó 35 câu hỏi tiếng Việt trên nguồn tiếng Anh. Câu tự viết không gán nhãn tay từng đoạn: đoạn đúng là mọi đoạn khớp bộ biểu thức chính quy của câu đó, nên có câu có hàng nghìn đoạn đúng. Hit@k và MRR vì vậy có ý nghĩa hơn Recall@k.

Bảng 5.2. Năm chế độ truy xuất và thí nghiệm cắt bỏ từng bộ khỏi RRF

Chế độ	Hit@1	Hit@5	MRR	nDCG@10	Trung vị (ms)
BM25	0,557	0,701	0,630	0,617	8
Vector e5	0,603	0,822	0,705	0,689	150
Đồ thị	0,391	0,517	0,445	0,424	258
Lai (RRF)	0,609	0,805	0,694	0,688	258
Lai + rerank	**0,724**	**0,856**	**0,783**	**0,756**	429
Bỏ BM25	0,598	0,782	0,684	0,672	–
Bỏ vector	0,603	0,764	0,675	0,654	–
Bỏ đồ thị	0,598	0,776	0,679	0,672	–

Bảng 5.2 và Hình 5.4 cho bốn nhận xét:

- Rerank mới là bước tạo khác biệt: MRR tăng từ 0,6943 lên 0,7830, đổi lại trung vị độ trễ từ 258,31 lên 428,61 ms.
- Riêng RRF không tốt hơn vector đơn. Bỏ từng bộ khỏi RRF chỉ làm MRR giảm khoảng 0,01 đến 0,02, nên đồ thị đóng góp nhỏ vào xếp hạng đoạn văn. Giá trị của nó nằm ở truy vấn nhiều bước và ở đường dẫn giải thích cho tác tử.
- Câu tiếng Việt trên nguồn tiếng Anh rất yếu: MRR chỉ 0,1714 với lai và 0,3607 với rerank. Đây là điểm yếu lớn nhất của kho hiện tại.



Hình 5.4. MRR theo nhóm câu hỏi: khó nhất là câu tiếng Việt trên nguồn tiếng Anh

- Số của nhóm tự viết lạc quan vì công thức điểm đồ thị được chỉnh trên chính nhóm này; câu sinh tự động thì có từ vựng gần đoạn gốc.

Ở khâu sinh câu trả lời, đồ án so sánh trên 30 câu cùng Qwen có và không có ngữ cảnh (Bảng 5.3). RAG cải thiện rõ, nhưng tỉ lệ chứa đúng đáp án vẫn dưới một nửa, và khi chỉ hỏi trực tiếp, mô hình hiếm khi tự trích dẫn. Vì thế tác tử Advisor có khuôn viết riêng và Critic kiểm trích dẫn ở Chương 6.

Bảng 5.3. Sinh câu trả lời có và không có RAG

Chỉ số	Có RAG	Không RAG
F1 theo token	0,3061	0,1914
Chứa đáp án vàng	0,4667	0,3000
Con số có trong ngữ cảnh	1,000	0,8561

Chương 6

Luật, học tăng cường và hệ đa tác tử

6.1 Hệ luật an toàn

Hệ luật giữ các quyết định không được phép phụ thuộc vào xác suất. Luật nằm trong tệp dữ liệu `rules.json`, mỗi luật có mã, nhóm, mức độ, ưu tiên, điều kiện và nguồn trích. Bộ máy suy diễn tiến chạy theo ưu tiên giảm dần; luật cháy có thể thêm dữ kiện để luật khác cháy ở vòng sau. Ngưỡng chẩn đoán được đọc từ cơ sở tri thức, nên một con số có nguồn chỉ tồn tại ở một nơi.

Bảng 6.1. Nhóm luật của RuleGuard

Nhóm	Số luật	Ví dụ
Khẩn cấp	12	Đường huyết rất cao, rất thấp, triệu chứng cấp
An toàn thuốc	11	Metformin với eGFR thấp, rượu, thuốc cản quang
Ngưỡng chẩn đoán	9	HbA1c, đường huyết lúc đói
Yếu tố nguy cơ	7	BMI, huyết áp, tuổi
Chính sách	5	Không nêu liều, không kê đơn
Tổng	**44**	

Ví dụ luật `MED-01`: người dùng đang dùng metformin và eGFR dưới 30 thì phát cảnh báo khẩn cấp. Khi chỉ có creatinin, tuổi, giới, hệ thống tự tính eGFR theo CKD-EPI 2021 rồi mới so khớp.

Đoạn mã 6.1. Vòng suy diễn tiến của hệ luật

```python
# dnx/rules.py
def evaluate(facts):
    """Trả các luật đã cháy, nặng trước rồi tới ưu tiên cao trước, đúng dạng trong hợp đồng."""
    f, fired = prepare(facts), {}
    for _ in range(MAX_ROUNDS):
        changed = False
        for rule in load_rules():
            if rule["id"] in fired:
                continue
            ok, seen = _fires(rule, f)
            if not ok:
                continue
            fired[rule["id"]] = (rule, seen)
            for k, v in rule.get("then", {}).items():
                changed = changed or f.get(k) != v
                f[k] = v
            changed = True
        if not changed:
            break
    rows = sorted(fired.values(), key=lambda x: (-SEVERITY_RANK[x[0]["severity"]], -x[0]["priority"]))
    return [{"rule_id": r["id"], "severity": r["severity"], "message": r["message"], "source": r["source"],
             "fired_on": seen, "group": r["group"]} for r, seen in rows]
```

6.2 Học tăng cường: sàng lọc thích ứng

6.2.1 Mô hình hóa

Một bảng câu hỏi cố định hỏi mọi người cùng 21 câu, dù nhiều câu không giúp gì cho người đó. Đồ án coi đây là MDP trên chính dữ liệu thật `brfss`: mỗi tập là một hồ sơ thật, đáp án bị che và chỉ lộ ra khi tác tử hỏi.

- **Trạng thái:** đáp án đã có, mặt nạ các câu đã hỏi, logit xác suất hiện tại.
- **Hành động:** hỏi một đặc trưng chưa hỏi, hoặc dừng. Hành động không hợp lệ bị che.
- **Thưởng:** $r_t = \Delta \text{log-score} - \lambda c_k$. Chi phí c_k phản ánh công sức của người trả lời: câu biết ngay rẻ, câu phải nhớ kết quả khám đắt. Phần log-score được chia theo từng bước bằng biến đổi thế năng nên không làm đổi chính sách tối ưu. Đây là giả định thiết kế, đổi bảng chi phí thì phải huấn luyện lại.

Xác suất do một MLP chịu được thiếu đặc trưng cung cấp: mạng nhận giá trị nhân mặt nạ nối với chính mặt nạ, huấn luyện với mặt nạ ngẫu nhiên, nên dự đoán được từ bất kỳ tập câu trả lời nào.

Đoạn mã 6.2. Mô hình dự đoán chịu thiếu đặc trưng

```python
# dnx/rl/predictor.py
class MaskedMLP(nn.Module):
    def __init__(self, n=N, hidden=HIDDEN, dropout=DROPOUT):
        super().__init__()
        layers, last = [], 2 * n
        for h in hidden:
            layers += [nn.Linear(last, h), nn.ReLU(),
             ↪  nn.Dropout(dropout)]
            last = h
        self.net = nn.Sequential(*layers, nn.Linear(last, 1))

    def forward(self, x, mask):
        return self.net(torch.cat([x * mask, mask],
         ↪  dim=-1)).squeeze(-1)

    def prob(self, x, mask):
        return torch.sigmoid(self(x, mask))
```

6.2.2 Huấn luyện Double DQN

Môi trường chạy song song hàng trăm hồ sơ trên GPU. Bộ nhớ phát lại nằm luôn trên GPU, mạng đích cập nhật mềm. Bước tính mục tiêu dưới đây là phần cốt lõi của Double DQN: mạng trực tuyến chọn hành động, mạng đích đánh giá.

Đoạn mã 6.3. Huấn luyện Double DQN song song trên GPU

```python
# dnx/rl/dqn.py
def train(model, X_tr, X_va, y_va, lam, seed, cfg=CFG, on_eval=None):
    """Huấn luyện một seed; trả trọng số tốt nhất theo mục tiêu trên
     ↪  tập kiểm định và lịch sử học."""
    dev = X_tr.device
    gen = torch.Generator(device=dev).manual_seed(seed)
    torch.manual_seed(seed)
    env = ScreenEnv(model, X_tr, cfg["n_env"], lam, gen)
    qnet, target = QNet().to(dev), None
    target = copy.deepcopy(qnet)
    opt = torch.optim.Adam(qnet.parameters(), lr=cfg["lr"])
    buf = Replay(cfg["replay"], dev)
    hist, best, best_j, losses, t0 = [], None, -1e9, [], time.time()
```

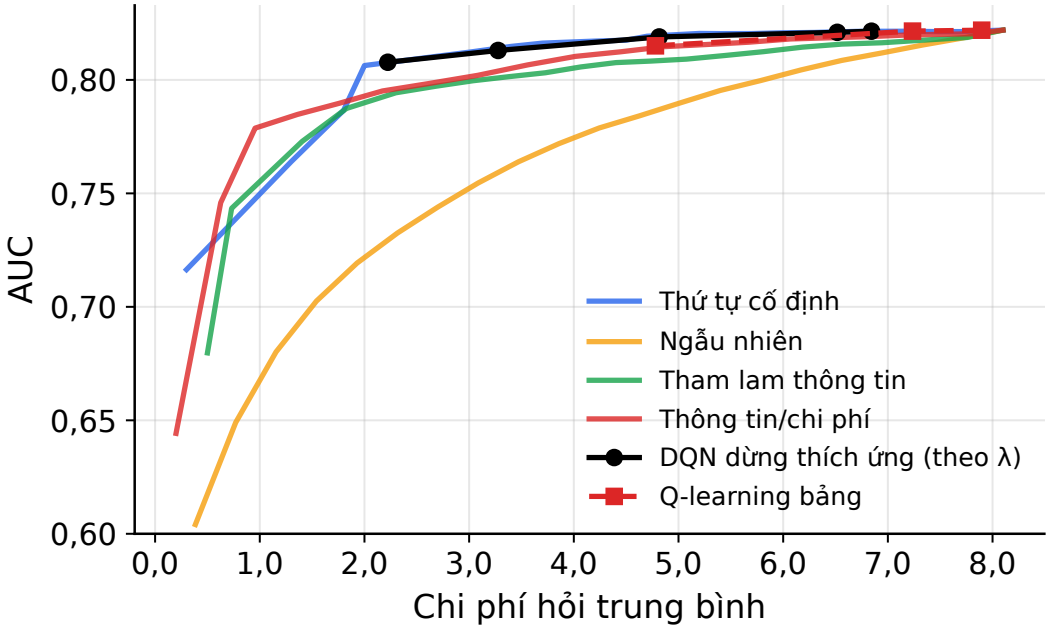
```
    decay = cfg["eps_frac"] * cfg["steps"]
    for t in range(1, cfg["steps"] + 1):
        eps = max(cfg["eps_end"], cfg["eps_start"] + (cfg["eps_end"] - cfg["eps_start"]) * t / decay)
        s = env.state()
        with torch.no_grad():
            a = masked_q(qnet(s), s).argmax(1)
        rand = torch.rand(env.n, device=dev, generator=gen) < eps
        if rand.any():
            noise = torch.rand(env.n, N + 1, device=dev, generator=gen).masked_fill(~valid_actions(s[:, N:2 * N]), -1)
            a[rand] = noise.argmax(1)[rand]
        r, done = env.step(a)
        buf.add(s, a, r, env.state(), done)
        env.reset_where(done)
        if t < cfg["warmup"]:
            continue
        bs, ba, br, bs2, bd = buf.sample(cfg["batch"], gen)
        with torch.no_grad():
            next_a = masked_q(qnet(bs2), bs2).argmax(1)
            next_q = masked_q(target(bs2), bs2).gather(1, next_a[:, None]).squeeze(1)
            y = br + cfg["gamma"] * (1 - bd) * next_q
        loss = F.smooth_l1_loss(qnet(bs).gather(1, ba[:, None]).squeeze(1), y)
        opt.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(qnet.parameters(), 10.0)
        opt.step()
        losses.append(loss.detach())
        with torch.no_grad():
            for p, q in zip(target.parameters(), qnet.parameters()):
                p.lerp_(q, cfg["tau"])
        if t % cfg["eval_every"] == 0 or t == cfg["steps"]:
            row = evaluate_objective(qnet, model, X_va, y_va, lam)
            row.update(step=t, eps=eps,
                       loss=float(torch.stack(losses).mean()),
                       seconds=round(time.time() - t0, 1))
            losses = []
            hist.append(row)
            if row["objective"] > best_j:
                best_j, best = row["objective"],
                    copy.deepcopy(qnet.state_dict())
            if on_eval:
                on_eval(row)
    return best, hist
```

6.2.3 Kết quả

Đồ án chấm trên 34.422 hồ sơ test bằng nhãn thật, so với hỏi đủ 21 câu, thứ tự cố định theo độ quan trọng, ngẫu nhiên, tham lam theo thông tin và Q-learning bảng. Mỗi mức phạt λ cho một điểm trên đường đánh đổi. Do giới hạn thời gian, $\lambda = 0{,}002$ chạy 5 seed, các mức còn lại chạy 2 seed; Q-learning bảng chỉ chạy ba mức λ, thua DQN ở cả ba.

Bảng 6.2. DQN dừng thích ứng theo λ trên test, so với AUC của hai đối chứng tại cùng chi phí

λ	Seed	Số câu	Chi phí	AUC DQN	F1	Cố định	Thông tin/phí
0,0005	2	17,3	6,84	0,8215	0,478	0,8214	0,8191
0,001	2	16,1	6,52	0,8210	0,478	0,8212	0,8187
0,002	5	11,3	4,82	0,8189	0,476	0,8195	0,8144
0,005	2	7,4	3,28	0,8130	0,468	0,8140	0,8039
0,01	2	4,5	2,22	0,8078	0,466	0,8076	0,7955

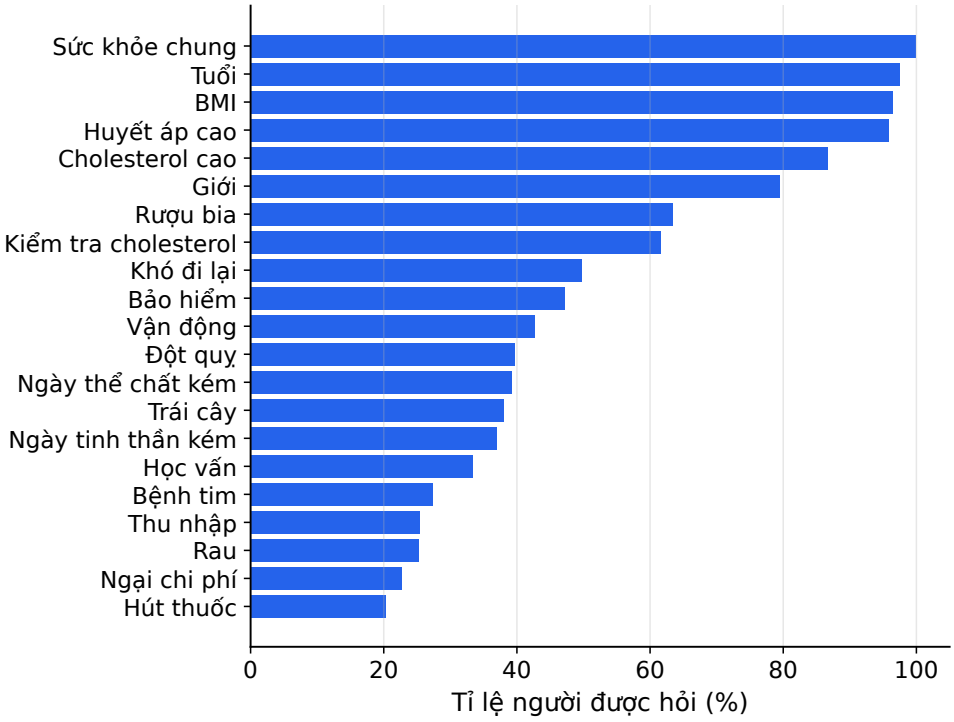


Hình 6.1. Đánh đổi giữa chi phí hỏi và AUC của các chính sách

Bảng 6.2 và Hình 6.1 cho ba kết luận. Thứ nhất, sàng lọc thích ứng tiết kiệm thật: với $\lambda = 0{,}002$, tác tử chỉ hỏi trung bình khoảng 11 câu thay vì 21, AUC giảm chưa tới 0,004. Thứ hai, DQN **không** vượt được thứ tự cố định theo độ quan trọng: tại cùng chi phí, hai bên chênh nhau không quá 0,001 ở mọi mức λ. Lý do hợp lý nhất là 21 câu của BRFSS tương quan với nhau và thứ tự cố định đã tính từ chính dữ liệu, nên dư địa cá nhân hóa nhỏ. Thứ ba, DQN cao hơn tham lam thông tin trên chi phí ở mọi mức λ, khoảng cách rõ nhất ở ngân sách thấp: 0,8078 so với 0,7955 với khoảng 4,5 câu.

Đóng góp thực tế của DQN vì vậy không phải là vượt thứ tự cố định, mà là tự quyết định khi nào dừng cho từng người: thứ tự cố định chỉ cho biết nên hỏi gì tiếp, không cho biết nên hỏi bao nhiêu câu. Mỗi bước cũng chỉ cần một lần suy luận, thay vì đánh giá lại mọi câu hỏi như chiến lược tham lam.

Hình 6.2 cho thấy chính sách thực sự thích ứng: sức khỏe chung, tuổi, BMI, huyết áp gần như luôn được hỏi, còn các câu khác chỉ hỏi với một phần người dùng.



Hình 6.2. Tỉ lệ người được hỏi từng câu dưới chính sách DQN

6.3 Hệ đa tác tử

6.3.1 Vai trò và đồ thị trạng thái

DiaNexus có tám vai trò chia sẻ một bảng đen `Board`. Mỗi tác tử chỉ giữ vài công cụ, nên mô hình ngôn ngữ nhỏ không phải chọn giữa quá nhiều lựa chọn (Bảng 6.3). Việc nào kiểm được bằng mã thì kiểm bằng mã, LLM Qwen2.5-7B [28] chỉ dùng cho phần cần sinh ngôn ngữ.

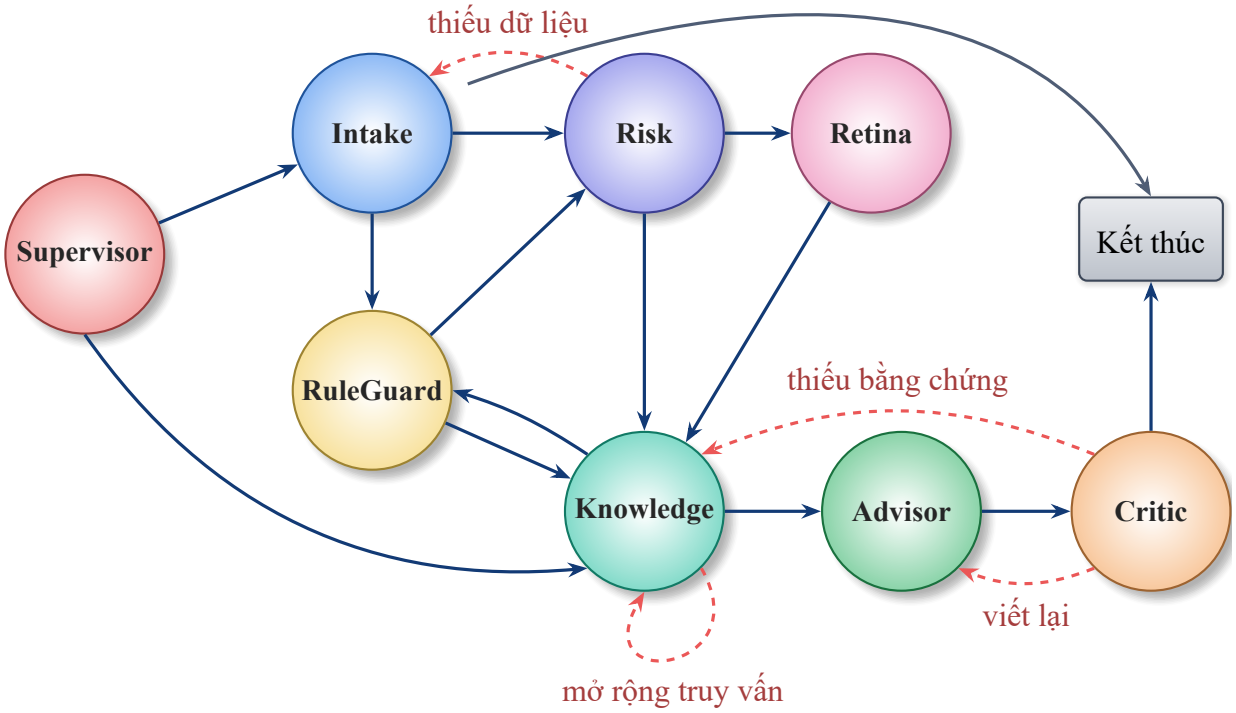
Luồng điều khiển là dữ liệu: mỗi nút có danh sách cạnh điều kiện, cạnh đầu tiên thỏa mãn được chọn (Hình 6.3). Các cạnh đỏ quay lui là điểm phân biệt hệ này với một đường ống: Risk trả về Intake khi thiếu dữ liệu, Knowledge tự mở rộng truy vấn khi bằng chứng yếu, Critic trả bản nháp về Advisor hoặc đòi thêm bằng chứng. Mỗi nút có giới hạn số lượt nên vòng lặp luôn dừng.

Đoạn mã 6.4. Đồ thị điều phối khai báo dưới dạng dữ liệu

```python
# dnx/agents/graph.py
GRAPH = {
    "supervisor": [(lambda b: b.intent == "out_of_scope", "advisor", "ngoài phạm vi"),
                   (lambda b: b.intent != "knowledge" or A.PERSONAL.search(b.query.lower()) is not None, "intake", "có người dùng"),
                   (lambda b: True, "knowledge", "hỏi kiến thức")],
    "intake": [(lambda b: _asked(b), END, "hỏi lại người dùng"),
               (lambda b: b.question is not None and b.risk is None, "risk", "RL dừng hỏi"),
               (_first_guard, "rule_guard", "kiểm khẩn cấp"),
               (_need_risk, "risk", "cần ước lượng"),
               (_need_retina, "retina", "có ảnh"),
               (lambda b: True, "knowledge", "tra cứu")],
    "rule_guard": [(lambda b: b.guard_runs >= 2 or bool(b.evidence), "advisor", "viết câu trả lời"),
                   (_need_risk, "risk", "cần ước lượng"),
                   (_need_retina, "retina", "có ảnh"),
                   (lambda b: True, "knowledge", "tra cứu")],
    "risk": [(lambda b: b.need_question, "intake", "thiếu dữ liệu, hỏi lại"),   # quay lui 1
             (_need_retina, "retina", "có ảnh"),
             (lambda b: True, "knowledge", "tra cứu")],
    "retina": [(lambda b: True, "knowledge", "tra cứu")],
    "knowledge": [(lambda b: b.weak_evidence, "knowledge", "mở rộng truy vấn"),  # quay lui 2
                  (_final_guard, "rule_guard", "soát luật"),
                  (lambda b: True, "advisor", "viết câu trả lời")],
    "advisor": [(_critic_on, "critic", "kiểm tra"),
                (lambda b: True, END, "trả lời")],
    "critic": [(lambda b: b.critic_ok, END, "đạt"),
               (lambda b: b.need_evidence, "knowledge", "thiếu bằng chứng"),     # quay lui 3
               (lambda b: True, "advisor", "viết lại")],
               # quay lui 4
}
```

Bảng 6.3. Tám vai trò của hệ đa tác tử

Tác tử	Nhiệm vụ	Công cụ
Supervisor	Xác định ý định, định tuyến	Luật, LLM
Intake	Trích hồ sơ từ tiếng Việt, hỏi lại	RL `next_question`
Risk	Ước lượng nguy cơ, SHAP	`predict`
Retina	Phân cấp ảnh đáy mắt	`grade`
Knowledge	Truy xuất lai, tự mở rộng truy vấn	`search`, `subgraph`
RuleGuard	Khẩn cấp, ngưỡng, an toàn thuốc	`evaluate`
Advisor	Viết câu trả lời có trích dẫn	LLM
Critic	Kiểm số, trích dẫn, liều thuốc	`check`



Hình 6.3. Đồ thị trạng thái của hệ đa tác tử, nét đỏ là cạnh quay lui

6.3.2 Critic kiểm bằng mã

Critic không nhờ LLM tự chấm. Nó kiểm năm loại lỗi: nêu liều thuốc, khẳng định người dùng đã mắc bệnh, con số không có trong dữ liệu đầu vào, trích dẫn không tồn tại, câu thiếu trích dẫn. Câu trả lời trượt được trả về Advisor kèm danh sách lỗi cụ thể.

6.3.3 Đánh giá bằng thí nghiệm cắt bỏ

Đồ án chạy 48 kịch bản thuộc tám nhóm qua năm cấu hình, tổng cộng 240 lượt có LLM thật (Bảng 6.4, Hình 6.4). Mỗi lượt được chấm bằng mã: định tuyến đúng, số

Đoạn mã 6.5. Critic kiểm câu trả lời bằng mã

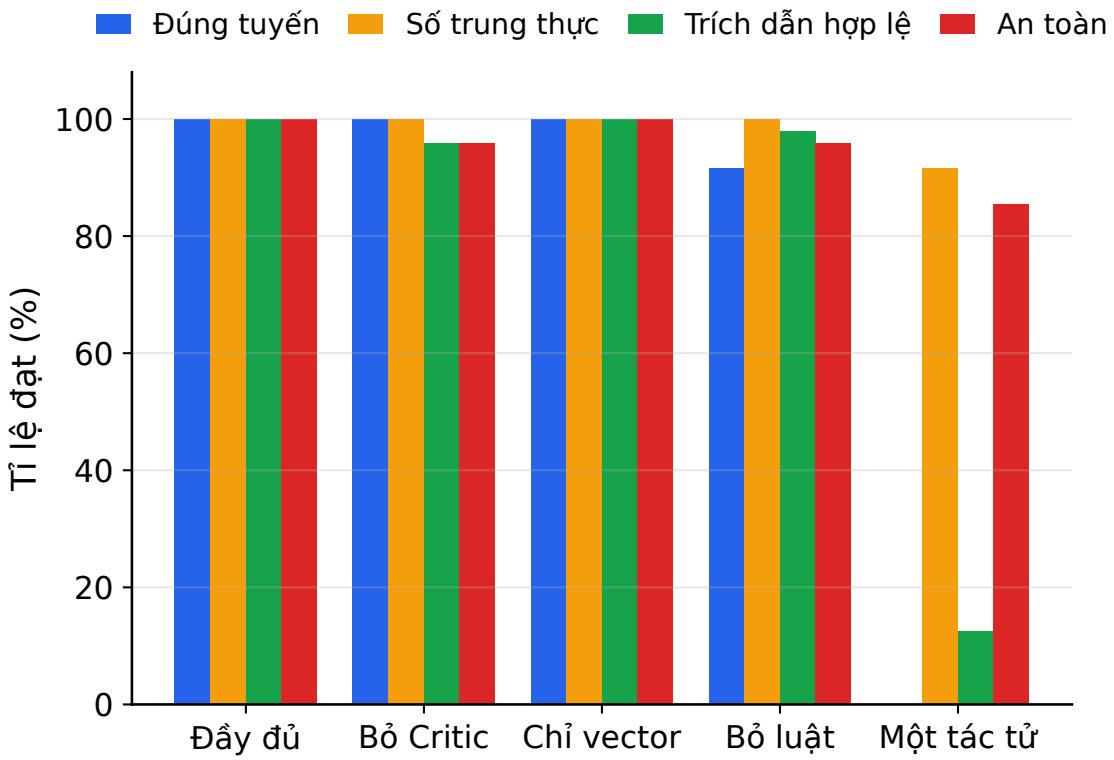
```python
# dnx/agents/textcheck.py
def check(answer, n_evidence, sources, need_citation=True,
    rule_ids=()):
    """Danh sách lỗi dạng {"kind", "detail"}; rỗng là đạt. answer là
        phần do LLM viết, chưa có câu miễn trừ."""
    problems = []
    hits = dose_hits(answer)
    if hits:
        problems.append({"kind": "dose", "detail": f"Không được nêu
            liều: {hits[:3]}"})
    claims = diagnosis_claims(answer)
    if claims:
        problems.append({"kind": "diagnosis", "detail": "Không được
            khẳng định người dùng đã mắc bệnh."})
    if wrong_range(answer, rule_ids):
        problems.append({"kind": "wrong_range", "detail": "Xét nghiệm
            đã ở ngưỡng chẩn đoán đái tháo đường, không được xếp vào
            tiền đái tháo đường."})
    nums = bad_numbers(answer, sources)
    if nums:
        problems.append({"kind": "number", "detail": f"Số không có
            trong dữ liệu: {nums[:5]}"})
    if need_citation:
        bad, uncited = citation_problems(answer, n_evidence)
        if bad:
            problems.append({"kind": "citation", "detail": f"Trích dẫn
                không tồn tại: {bad[:5]}"})
        if not CITE.search(answer):
            problems.append({"kind": "no_citation", "detail": "Không có
                trích dẫn [n] nào."})
        elif uncited:
            problems.append({"kind": "uncited", "detail":
                f"{len(uncited)} câu chưa có [n]"})
    return problems
```

có trong dữ liệu, trích dẫn hợp lệ và an toàn, tức không nêu liều, không bỏ sót ca khẩn cấp, không trả lời ngoài phạm vi.

Kết quả rõ nhất nằm ở an toàn. Bỏ hệ luật làm hai ca khẩn cấp bị bỏ sót: trên 6 kịch bản khẩn cấp, tỉ lệ an toàn giảm từ 1,000 xuống 0,6667. Cũng ở cấu hình này, hai câu trả lời xếp sai nhóm xét nghiệm. Bỏ Critic làm hai câu trả lời nêu liều thuốc lọt ra ngoài, dù tiết kiệm được gần hai phần ba thời gian. Cấu hình một tác tử kém nhất, tỉ lệ trích dẫn hợp lệ chỉ 0,1250, cho thấy một lời gọi LLM với đủ công cụ không thay được việc chia vai trò.

Bảng 6.4. Thí nghiệm cắt bỏ trên 48 kịch bản, mỗi cấu hình một hàng

Cấu hình	Định tuyến	Số đúng	Trích dẫn	An toàn	Giây
Đầy đủ	1,000	1,000	1,000	1,000	10,0
Bỏ Critic	1,000	1,000	0,958	0,958	3,5
Chỉ vector	1,000	0,979	1,000	1,000	8,7
Bỏ luật	0,917	1,000	0,979	0,917	8,9
Một tác tử	–	0,938	0,125	0,896	5,5



Hình 6.4. Tỉ lệ đạt theo từng tiêu chí của năm cấu hình

Bảng này cần đọc thận trọng ở ba điểm:

- Điểm tuyệt đối của cấu hình đầy đủ phần lớn đến từ cách đo: Critic và bộ chấm dùng chung một bộ kiểm, và 22 trên 48 lượt phải cắt câu sau hai vòng viết lại.
- Cấu hình chỉ vector gần như bằng cấu hình đầy đủ, chỉ kém nhẹ ở cột số đúng. Thước đo không đủ nhạy với chất lượng truy xuất, nên không thể kết luận đồ thị vô ích từ bảng này.
- Bộ chấm chưa kiểm được đúng sai y khoa nói chung, chỉ kiểm được những gì có luật đối chiếu. Đây là khoảng trống lớn nhất của hệ thống.

Quá trình đánh giá phát hiện bốn lỗi thật. Câu "đột ngột méo miệng, yếu nửa người" từng bị coi là ngoài phạm vi, nên Supervisor được sửa để chạy luật khẩn cấp trước khi từ chối. Chỉ số bỏ sót khẩn cấp ban đầu không bao giờ báo đỏ, vì từ

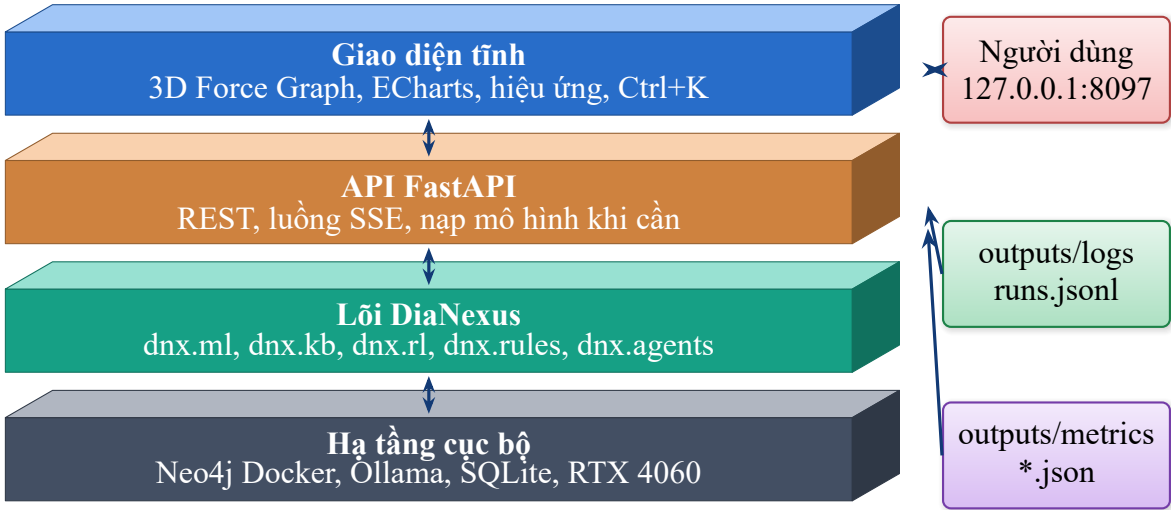
"bác sĩ" luôn có trong câu miễn trừ; câu miễn trừ nay được bỏ khi chấm. Nghiêm trọng nhất, LLM từng xếp HbA1c 6,6% vào tiền đái tháo đường dù luật đã báo ở ngưỡng chẩn đoán, và Critic lẫn bộ chấm đều không thấy. Đồ án thêm phép kiểm `wrong_range` đối chiếu câu trả lời với luật đã cháy, rồi chạy lại mọi kịch bản liên quan ở cả năm cấu hình. Khi thử trực tiếp còn lộ ra lỗi định tuyến: câu nêu số xét nghiệm nhưng không có từ xưng hô bị coi là hỏi kiến thức chung, nên luật không có dữ liệu để chạy; "đường huyết đói" thiếu chữ "lúc" cũng không được nhận là lúc đói. Cả hai đã được sửa, kèm kiểm thử. Bảng 6.4 là số sau tất cả các lần sửa. Bài học là điểm 1,000 trước đó đã không chứng minh được câu trả lời đúng, chỉ chứng minh nó qua được các phép kiểm lúc đó.

Chương 7

Website DiaNexus

7.1 Kiến trúc triển khai

Sản phẩm chạy hoàn toàn trên localhost tại `127.0.0.1:8097`, gồm máy chủ FastAPI và giao diện tĩnh, không dùng CDN nên không có dữ liệu nào rời khỏi máy. Hình 7.1 là bốn tầng của hệ thống. Khởi động máy chủ không nạp mô hình, mô hình chỉ được nạp ở yêu cầu đầu tiên cần đến nó.



Hình 7.1. Bốn tầng triển khai cục bộ của DiaNexus

Máy chỉ có khoảng 47 GB bộ nhớ commit nên website phải tự bảo vệ: mỗi lúc chỉ một yêu cầu nặng được chạy, lần nạp mô hình đầu tiên phải đặt chỗ bộ nhớ trước. Khi không đủ bộ nhớ, API trả 503 với thông báo rõ ràng thay vì làm sập máy.

Đoạn mã 7.1. Cổng giới hạn yêu cầu nặng của web

```
# web/api/lazy.py
@contextmanager
def heavy_turn(key, gb, what):
    """Mỗi lúc một yêu cầu nặng. Lần nạp mô hình đầu đặt chỗ commit qua dnx.mem.slot; thiếu commit thì trả 503 ngay
    chứ không treo yêu cầu, vì slot() chờ vô hạn còn nạp liều sẽ làm cả máy sập."""
    if not _heavy.acquire(timeout=TURN_WAIT_S):
        raise Unavailable(f"Đang bận tác vụ nặng khác, chưa chạy được {what}.")
    try:
        if key in _loaded:
            yield
            return
        free = free_commit_gb() - _reserved_gb()  # tính cả phần các làn đã đặt chỗ, nếu không slot() sẽ chờ vô hạn
        if free < gb + COMMIT_MARGIN_GB:
            raise Unavailable(f"Chờ bộ nhớ để nạp {what}: cần {gb} GB commit, đang trống {free:.1f} GB.")
        with slot(gb, f"web {what}"):
            yield
        _loaded.add(key)
    finally:
        _heavy.release()
```

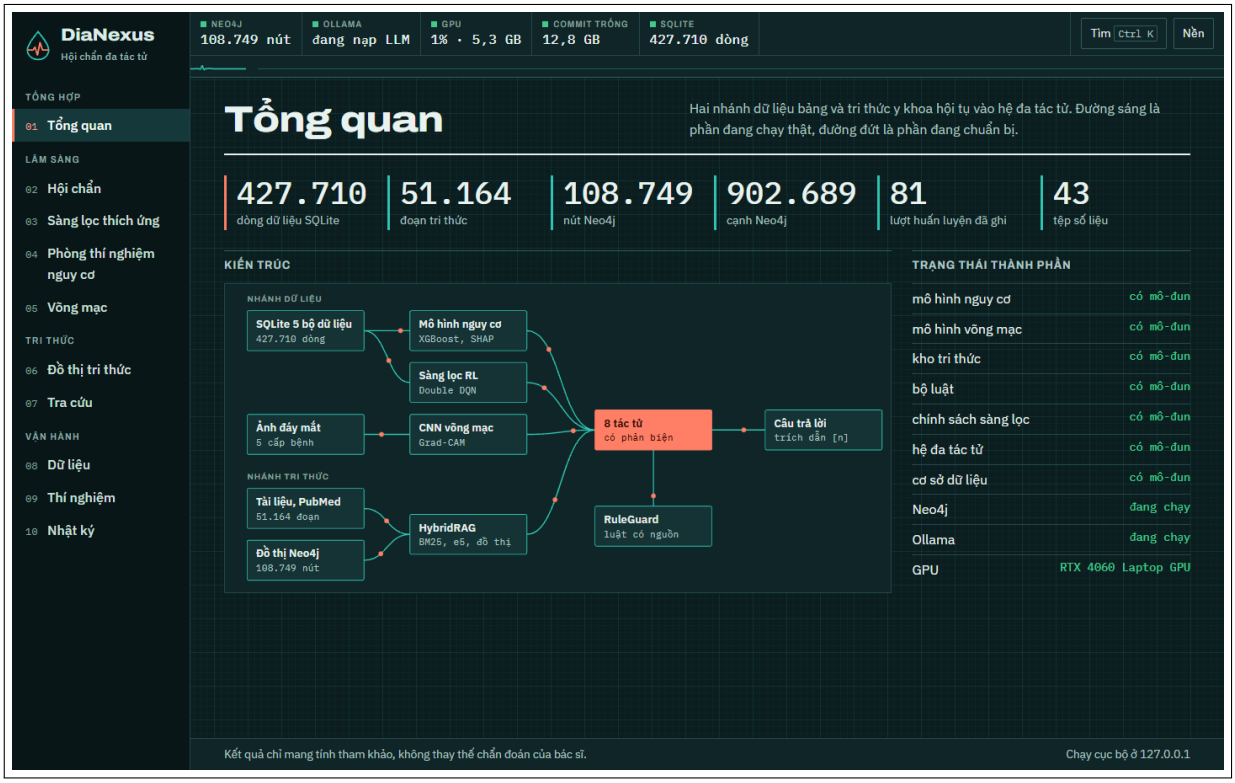
7.2 Các trang chức năng

Giao diện lấy cảm hứng từ màn hình theo dõi đầu giường bệnh: nền tối xanh ngọc, điểm nhấn san hô, thanh sinh hiệu trên cùng hiện trạng thái thật của Neo4j, Ollama, GPU, bộ nhớ và CSDL. Đường tín hiệu chạy theo tải GPU và phẳng đi khi mất kết nối. Mười trang chia theo bốn nhóm: tổng hợp, lâm sàng, tri thức và vận hành. Trang Tổng quan (Hình 7.2) cho thấy hai nhánh dữ liệu và tri thức hội tụ về hệ tác tử bằng số liệu sống.

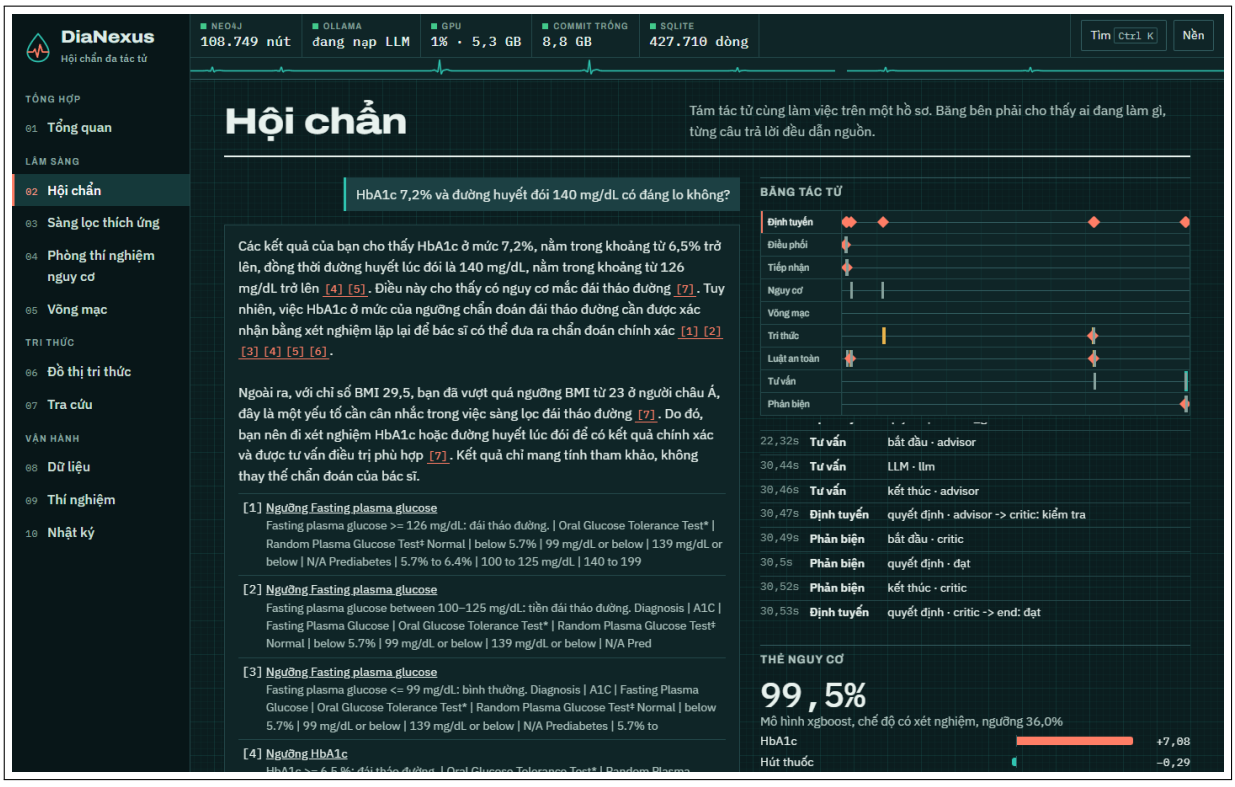
Trang Hội chẩn (Hình 7.3) là nơi thể hiện rõ nhất tính đa tác tử. Mỗi sự kiện của tác tử được đẩy về trình duyệt qua Server-Sent Events, người dùng thấy lần lượt Supervisor định tuyến, Knowledge truy xuất, Critic kiểm tra, và có thể bấm vào trích dẫn `[n]` để xem nguồn.

Trang Sàng lọc thích ứng (Hình 7.4) là tính năng mới của đồ án: mỗi câu hỏi do chính sách RL chọn, thước xác suất nhảy theo từng đáp án và hiển thị giá trị Q của các câu hỏi ứng viên.

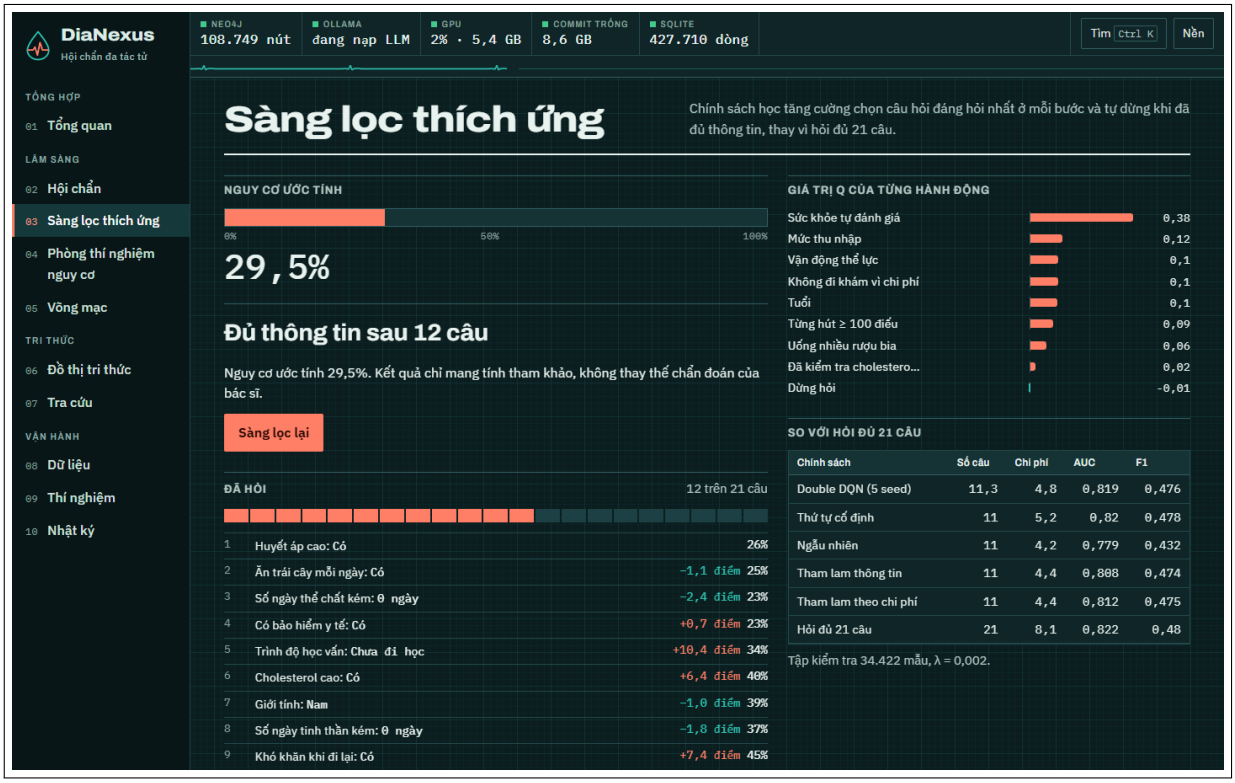
Trang Phòng thí nghiệm nguy cơ (Hình 7.5) cho kéo thanh trượt để thấy nguy



Hình 7.2. Trang Tổng quan: số liệu sống và sơ đồ hai nhánh



Hình 7.3. Trang Hội chẩn: dòng thời gian tác tử trực tiếp và trích dẫn



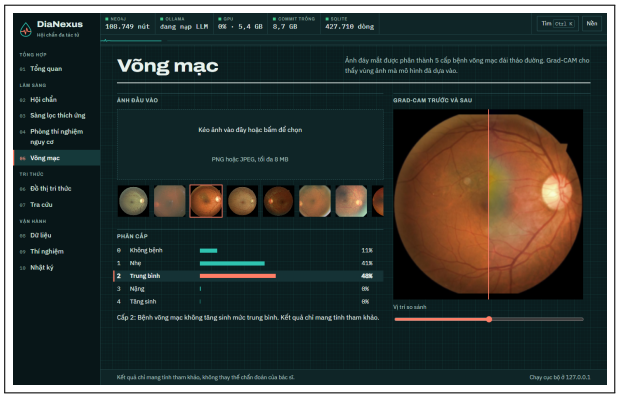
Hình 7.4. Trang Sàng lọc thích ứng: chính sách RL chọn từng câu hỏi

cơ đổi ngay lập tức. Tính năng song sinh số đặt hồ sơ hiện tại cạnh hồ sơ sau khi đạt mục tiêu, kèm SHAP và các luật đã kích hoạt.



Hình 7.5. Phòng thí nghiệm nguy cơ: what-if, SHAP và song sinh số

Trang Võng mạc (Hình 7.6) có thanh trượt so sánh ảnh gốc với Grad-CAM. Trang Đồ thị tri thức (cùng Hình 7.6) đọc trực tiếp từ Neo4j, vẽ không gian 3D, bấm vào nút để mở rộng lân cận và hiện đúng câu Cypher đã chạy.



(a) Võng mạc: phân cấp và Grad-CAM **(b)** Đồ thị tri thức 3D từ Neo4j

Hình 7.6. Hai trang trực quan của DiaNexus

Trang Dữ liệu duyệt bảng SQLite theo phân trang, đọc qua kết nối chỉ đọc. Trang Thí nghiệm so sánh mọi lượt chạy từ nhật ký. Trang Nhật ký hiển thị log thật theo thời gian thực và phát lại vết tác tử đã lưu.



(a) Duyệt CSDL và thống kê cột **(b)** Log trực tiếp và phát lại vết

Hình 7.7. Hai trang vận hành: dữ liệu và nhật ký

7.3 Kiểm thử và an toàn

Website được kiểm bằng trình duyệt tự động ở hai cỡ màn hình 1440 và 390 px, ghi lỗi console và độ tràn ngang của từng trang vào tệp kết quả. API chỉ nghe ở địa chỉ cục bộ, SQLite mở ở chế độ chỉ đọc, tên bảng đi qua danh sách trắng, truy vấn dùng tham số, ảnh tải lên giới hạn 8 MB và kiểm chữ ký tệp, nên bảng tùy ý hoặc tệp không phải ảnh đều bị từ chối.

Chương 8
Kết luận và hướng phát triển

8.1 Kết quả đạt được

Đồ án đã hoàn thành DiaNexus, một hệ đa tác tử chạy cục bộ, phủ đủ các khối trong sơ đồ đề bài. CSDL SQLite gồm năm nguồn với 427.710 dòng, kèm 35.126 ảnh võng mạc. Sáu mô hình được so sánh trên ba bài toán, và XGBoost đạt ROC-AUC 0,8217 ở bài sàng lọc không cần xét nghiệm. Đồ thị tri thức trên Neo4j có 108.749 nút và 902.689 cạnh, là nền cho truy xuất lai. Chính sách Double DQN giảm số câu hỏi sàng lọc gần một nửa với mức giảm AUC rất nhỏ. Tám vai trò phối hợp qua đồ thị trạng thái có cạnh quay lui, được bảo vệ bởi hệ luật và Critic kiểm bằng mã. Website localhost có mười trang, thể hiện vết chạy của tác tử theo thời gian thực.

Về kiểm thử, 80 ca của `pytest` đều đạt, và thí nghiệm cắt bỏ 240 lượt cho thấy hệ luật và Critic giữ vai trò an toàn thật: bỏ chúng thì ca khẩn cấp bị bỏ sót hoặc liều thuốc lọt ra câu trả lời.

8.2 Những gì đồ án cho thấy

Bốn kết quả đáng chú ý nhất lại là những kết quả không như kỳ vọng ban đầu:

- Mô hình sâu không thắng XGBoost trên dữ liệu bảng, dù tốn thời gian hơn nhiều.
- DQN không vượt thứ tự câu hỏi cố định về AUC. Giá trị của nó nằm ở việc tự dừng và chi phí suy luận thấp.
- Isotonic không cải thiện hiệu chỉnh ở hai trên ba bài toán, vì XGBoost vốn đã khá chuẩn.
- RRF không vượt vector đơn; chỉ rerank mới cải thiện rõ truy xuất.

Các kết quả này được giữ nguyên trong báo cáo vì chúng cho biết nên đầu tư vào đâu ở bước sau.

8.3 Giới hạn

- Nhãn của bộ `patients_lab` gần như sinh theo ngưỡng, nên ROC-AUC cao ở bài có xét nghiệm không phản ánh năng lực thực tế.

- Mô hình võng mạc yếu ở cấp 1 và bộ ảnh không có mã bệnh nhân, nên kết quả có thể lạc quan.
- Bảng chi phí câu hỏi của RL là giả định thiết kế. Các mức λ ngoài $0{,}002$ chỉ có 2 seed.
- Mô hình huấn luyện trên dữ liệu nước ngoài; thí nghiệm chéo miền cho thấy xác suất lệch mạnh khi đổi quần thể, và đồ án chưa có dữ liệu người Việt để hiệu chỉnh lại.
- Phần lớn tri thức là tiếng Anh, câu hỏi tiếng Việt trên nguồn tiếng Anh truy xuất còn yếu. Hai tài liệu của Bộ Y tế trong kho là bản 2011 và 2017, chưa có hướng dẫn 5481/QĐ-BYT năm 2020.
- Cạnh do LLM trích có precision chỉ khoảng một nửa, và mẫu chấm do tác tử AI chấm, chưa có chuyên gia kiểm.
- Chưa có bước kiểm đúng sai y khoa trong câu trả lời của LLM, chỉ kiểm được hình thức, số và trích dẫn.
- Máy cá nhân chỉ phục vụ được một yêu cầu nặng mỗi lúc, nên hệ thống là bản trình diễn cho một người dùng, chưa phải dịch vụ.

8.4 Hướng phát triển

- Thu thập dữ liệu người Việt để hiệu chỉnh lại xác suất và kiểm chứng độ tin cậy.
- Tăng độ phân giải ảnh võng mạc và chia tập theo bệnh nhân bằng bộ dữ liệu có mã định danh.
- Mở rộng kho tri thức tiếng Việt và mở rộng trích xuất quan hệ bằng LLM ra toàn bộ kho văn bản.
- Thử RL với mô hình giá trị thông tin có học, thay vì chỉ học từ phần thưởng log-score.
- Đánh giá với bác sĩ và người dùng thật trước khi nghĩ đến bất kỳ ứng dụng lâm sàng nào.

Tài liệu tham khảo

[1] International Diabetes Federation. *IDF Diabetes Atlas*. 11th ed. Brussels: International Diabetes Federation, 2025.

[2] American Diabetes Association Professional Practice Committee. "2. Diagnosis and Classification of Diabetes: Standards of Care in Diabetes–2025". In: *Diabetes Care* 48.Suppl. 1 (2025), S27–S49.

[3] Varun Gulshan et al. "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs". In: *JAMA* 316.22 (2016), pp. 2402–2410.

[4] Leo Breiman. "Random Forests". In: *Machine Learning* 45.1 (2001), pp. 5–32.

[5] Tianqi Chen and Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System". In: *Proc. KDD*. 2016, pp. 785–794.

[6] Bianca Zadrozny and Charles Elkan. "Transforming Classifier Scores into Accurate Multiclass Probability Estimates". In: *Proc. KDD*. 2002, pp. 694–699.

[7] Chuan Guo et al. "On Calibration of Modern Neural Networks". In: *Proc. ICML*. 2017, pp. 1321–1330.

[8] Scott M. Lundberg and Su-In Lee. "A Unified Approach to Interpreting Model Predictions". In: *Proc. NeurIPS*. 2017.

[9] Yury Gorishniy et al. "Revisiting Deep Learning Models for Tabular Data". In: *Proc. NeurIPS*. 2021.

[10] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *Proc. CVPR*. 2016, pp. 770–778.

[11] Ramprasaath R. Selvaraju et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization". In: *Proc. ICCV*. 2017, pp. 618–626.

[12] Aidan Hogan et al. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37.

[13] Patrick Lewis, Ethan Perez, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Proc. NeurIPS*. 2020.

[14] Stephen Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.

[15] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proc. SIGIR*. 2009, pp. 758–759.

[16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.

[17] Christopher J. C. H. Watkins and Peter Dayan. "Q-Learning". In: *Machine Learning* 8 (1992), pp. 279–292.

[18] Volodymyr Mnih et al. "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518 (2015), pp. 529–533.

[19] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-Learning". In: *Proc. AAAI*. 2016, pp. 2094–2100.

[20] Jaromir Janisch, Tomas Pevny, and Viliam Lisy. "Classification with Costly Features Using Deep Reinforcement Learning". In: *Proc. AAAI*. 2019, pp. 3959–3966.

[21] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.

[22] Shunyu Yao, Hugh Zhao, et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *Proc. ICLR*. 2023.

[23] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd ed. Wiley, 2009.

[24] Beata Strack et al. “Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records”. In: *BioMed Research International* 2014 (2014), p. 781670.

[25] Jack W. Smith et al. “Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus”. In: *Proc. Annual Symposium on Computer Application in Medical Care*. 1988, pp. 261–265.

[26] Daniel Scott Himmelstein et al. “Systematic Integration of Biomedical Knowledge Prioritizes Drugs for Repurposing”. In: *eLife* 6 (2017), e26726.

[27] Zhichun Wang et al. *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672. 2024.

[28] Qwen Team. *Qwen2.5 Technical Report*. arXiv:2412.15115. 2024.